Appunti

Machine Learning

Insegnamento di Machine Learning
Docente: Prof. Valerio Freschi

Appunti redatti da

Leonardo Malaventura

Università degli Studi di Urbino Carlo Bo
Laurea magistrale in Informatica e Innovazione Digitale



Anno accademico 2026–2027

Appunti di studio non ufficiali, in aggiornamento.

Indice

1 Introduzione al Machine Learning **1**
1.1 Machine Learning e Intelligenza Artificiale 1
1.2 Apprendimento dai dati 1
1.3 Perché utilizzare il Machine Learning 1
1.4 Diffusione del Machine Learning 1

2 Rappresentazione dei dati **3**
2.1 Caratteristiche e spazio delle caratteristiche 3

3 Principali paradigmi di apprendimento **4**
3.1 Apprendimento supervisionato 4
3.2 Apprendimento non supervisionato 5
3.3 Reinforcement Learning 5

4 Modelli e parametri **6**
4.1 Classificatori lineari 6

5 Addestramento come problema di ottimizzazione **8**
5.1 Minimi e massimi 8
5.2 Trasformazione di un massimo in un minimo 9
5.3 Condizioni di ottimalità di ordine zero 9
5.4 Esempi 9

6 Convessità e difficoltà dell'ottimizzazione **11**

7 Ottimizzazione globale **12**
7.1 Maledizione della dimensionalità 12
7.2 Campionamento casuale 13

8 Ottimizzazione locale **14**
8.1 Idea generale dell'ottimizzazione locale 14
8.2 Lunghezza del passo 14
8.3 Norme vettoriali 15
8.4 Normalizzazione della direzione 15
8.5 Learning rate e iperparametri 16
8.6 Significato del learning rate 16
8.7 Regola del passo decrescente 16
8.8 Random search 17
8.9 Esempio di random search 17
8.10 Limiti del random search e dimensionalità 18
8.11 Ricerca per coordinate 18
8.12 Discesa per coordinate 19
8.13 History plot della funzione costo 19
8.14 Ottimizzazione locale e funzioni non convesse 20

9 Ottimizzazione del primo ordine e discesa del gradiente **21**
9.1 Gradiente e punti stazionari 21
9.2 Approssimazione lineare tramite Taylor 22
9.3 Direzione di massima salita e massima discesa 23
9.4 Gradiente e curve di livello 24

9.5 Algoritmo di discesa del gradiente . . . 24
9.6 Scelta del learning rate nella gradient descent . . . 26
9.7 Criteri di convergenza . . . 27
9.8 Perché utilizzare il gradiente . . . 27
9.9 Limiti della discesa del gradiente . . . 27

10 Ottimizzazione del secondo ordine . . . 29
10.1 Derivate seconde e matrice Hessiana . . . 29
10.2 Richiamo su autovalori e autovettori . . . 30
10.3 Convessità e condizioni di ottimalità del secondo ordine . . . 30
10.4 Funzioni quadratiche e approssimazione di Taylor . . . 31
10.5 Il metodo di Newton: dall'approssimazione all'algoritmo . . . 32

11 Regressione lineare e funzione di costo ai minimi quadrati . . . 36
11.1 Dai dati a una previsione . . . 36
11.2 Modello e ruolo dei parametri . . . 36
11.3 Errore di previsione e minimi quadrati . . . 37
11.4 Convessità e collegamento con le equazioni normali . . . 38
11.5 Outlier e funzione di costo alle deviazioni assolute . . . 39
11.6 Regressione pesata . . . 40

12 Classificazione lineare binaria e regressione logistica . . . 42
12.1 Dal punteggio lineare alla funzione gradino . . . 42
12.2 La sigmoide e il modello di regressione logistica . . . 43
12.3 Perché il costo ai minimi quadrati non è sempre la scelta migliore . . . 44
12.4 Cross-entropy: una funzione di costo adatta alla classificazione binaria . . . 44
12.5 Forma softplus ed etichette meno uno e più uno . . . 45
12.6 Tangente iperbolica e rappresentazione binaria simmetrica . . . 45

13 Generalizzazione e validazione . . . 47
13.1 Generalizzazione . . . 47
13.2 Training set, validation set e test set . . . 47
13.3 Predizione e metriche di valutazione per la regressione . . . 47

14 Pipeline di base di un sistema di Machine Learning . . . 49
14.1 Miglioramento delle prestazioni . . . 49

15 Laboratorio: notebook e richiami di Python . . . 50
15.1 Notebook, Jupyter e Google Colab . . . 50
15.2 Elementi essenziali del linguaggio . . . 50
15.3 Strutture dati e iterazione . . . 51
15.4 Funzioni e classi . . . 52
15.5 Uso del materiale nelle esercitazioni successive . . . 53

16 NumPy: rappresentazione e calcolo su array . . . 54
16.1 Gli array e la loro forma . . . 54
16.2 Indicizzazione, slicing e copie . . . 55
16.3 Tipi numerici e operazioni sugli elementi . . . 55
16.4 Prodotti matriciali e operazioni di algebra lineare . . . 56

Capitolo 1

Introduzione al Machine Learning

1.1 Machine Learning e Intelligenza Artificiale

Il **Machine Learning** (ML), traducibile come *apprendimento automatico*, è un sottoinsieme dell'**Intelligenza Artificiale** (*Artificial Intelligence*, AI).

L'Intelligenza Artificiale non coincide quindi con il Machine Learning: il Machine Learning rappresenta uno degli approcci utilizzabili per realizzare sistemi di Intelligenza Artificiale.

Storicamente, uno degli obiettivi dell'Intelligenza Artificiale è stato quello di realizzare sistemi capaci di riprodurre alcune forme di ragionamento e comportamento tipiche degli organismi intelligenti.

1.2 Apprendimento dai dati

Il Machine Learning utilizza i dati per individuare automaticamente **regolarità**, **relazioni** e **pattern**.

In un approccio algoritmico tradizionale, il programmatore specifica esplicitamente le regole che il sistema deve seguire.

Nel Machine Learning, invece, si forniscono al sistema dei dati dai quali un algoritmo può apprendere automaticamente un modello.

Il principio fondamentale può essere espresso nel seguente modo:

> Un algoritmo di Machine Learning apprende automaticamente dai dati.

Il problema si sposta quindi dalla definizione manuale di tutte le regole necessarie alla scelta e all'addestramento di un modello matematico in grado di rappresentare adeguatamente il problema.

1.3 Perché utilizzare il Machine Learning

Esistono problemi per i quali costruire manualmente una soluzione algoritmica risulta estremamente complesso.

Alcuni esempi sono:

- il riconoscimento di oggetti nelle immagini (*object recognition*);
- la comprensione del linguaggio parlato (*speech understanding*).

In problemi di questo tipo può essere difficile individuare esplicitamente tutte le regole necessarie per gestire correttamente ogni possibile caso.

Il Machine Learning può quindi essere utilizzato quando:

- il problema è troppo complesso per una soluzione algoritmica esplicita;
- si vuole ottenere un determinato comportamento a partire dai dati;
- il sistema deve essere capace di adattarsi alle informazioni osservate.

1.4 Diffusione del Machine Learning

La crescita del Machine Learning è stata favorita principalmente da tre fattori:

- lo sviluppo di algoritmi di apprendimento sempre più efficaci;
- la crescente disponibilità di dati;
- l'aumento della capacità computazionale dell'hardware.

Capitolo 2

Rappresentazione dei dati

2.1 Caratteristiche e spazio delle caratteristiche

Per essere utilizzato da un algoritmo di Machine Learning, un dato viene rappresentato mediante un insieme di **caratteristiche**, chiamate anche *features*.

Un dato può quindi essere rappresentato mediante un vettore:

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix},$$

dove ogni componente x_i rappresenta una caratteristica del dato.

L'insieme delle possibili rappresentazioni dei dati prende il nome di **spazio delle caratteristiche** (*feature space*).

Quando vengono considerate solamente due caratteristiche, ogni dato può essere rappresentato geometricamente come un punto in un piano cartesiano.

È quindi possibile interpretare alcuni problemi di Machine Learning in termini geometrici, studiando la disposizione dei dati nello spazio delle caratteristiche.

Capitolo 3

Principali paradigmi di apprendimento

Gli algoritmi di Machine Learning possono essere classificati in diversi paradigmi in base al tipo di informazione disponibile durante l'apprendimento e all'obiettivo del sistema.

3.1 Apprendimento supervisionato

Nell'**apprendimento supervisionato** (*supervised learning*) vengono forniti esempi etichettati che specificano il comportamento corretto del sistema.

L'obiettivo consiste nell'apprendere una relazione tra input e output:

$$f : X \to Y,$$

in modo che, dato un input x, il modello possa produrre un output:

$$y = f(x).$$

Due importanti categorie di problemi di apprendimento supervisionato sono:

- la **classificazione**;
- la **regressione**.

3.1.1 Classificazione

Nei problemi di classificazione l'output appartiene a un insieme **discreto** di categorie.

Ad esempio, in un problema di classificazione binaria:

$$y \in \{0, 1\}.$$

Il modello deve quindi stabilire a quale delle possibili classi appartenga un determinato dato.

Esempi di problemi di classificazione sono il riconoscimento di oggetti, la rilevazione di spam, la diagnosi assistita, il riconoscimento di cifre scritte a mano e il rilevamento di frodi.

3.1.2 Regressione

Nei problemi di regressione l'output assume invece un valore **continuo**.

Ad esempio:

$$y \in \mathbb{R}.$$

Un modello potrebbe, per esempio, utilizzare alcune caratteristiche di un'abitazione per produrre una stima del suo prezzo.

Un caso fondamentale è la regressione lineare, nella quale la predizione è una combinazione affine delle caratteristiche. Il modello e la relativa funzione di costo vengono sviluppati nella Sezione 11.

La differenza fondamentale è quindi:

$$\text{Classificazione} \quad \longrightarrow \quad \text{output discreto},$$

$$\text{Regressione} \quad \longrightarrow \quad \text{output continuo.}$$

3.2 Apprendimento non supervisionato

Nell'**apprendimento non supervisionato** (*unsupervised learning*) non vengono forniti esempi etichettati che specifichino l'output corretto.

L'obiettivo consiste invece nell'individuare automaticamente strutture, regolarità o rappresentazioni significative presenti nei dati di input.

Due esempi importanti sono:

- la **riduzione della dimensionalità** (*dimensionality reduction*), con cui si cerca una rappresentazione dei dati in uno spazio di dimensione inferiore;
- il **clustering**, che cerca di individuare gruppi di dati caratterizzati da proprietà o strutture simili.

3.3 Reinforcement Learning

Nel **Reinforcement Learning** il sistema riceve un **segnale di ricompensa** (*reward*) e apprende un comportamento con l'obiettivo di massimizzare tale ricompensa.

A differenza dell'apprendimento supervisionato, non vengono quindi fornite direttamente coppie input–output che descrivono la risposta corretta.

Capitolo 4

Modelli e parametri

Un modello di Machine Learning è caratterizzato da un insieme di **parametri**, i cui valori determinano il comportamento del modello.

Indichiamo il vettore dei parametri con

$$\mathbf{w} = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_n \end{bmatrix}.$$

È importante distinguere:

- $\mathbf{x}$, che rappresenta le **caratteristiche del dato**;
- $\mathbf{w}$, che rappresenta i **parametri del modello**.

L'addestramento consiste nel determinare opportuni valori dei parametri $\mathbf{w}$ utilizzando i dati disponibili.

4.1 Classificatori lineari

Un classificatore viene detto **lineare** quando la decisione tra le classi dipende da una funzione affine delle caratteristiche. Nel caso binario, il confine di separazione è definito dall'insieme dei punti in cui questa funzione si annulla.

Nel caso bidimensionale il **confine decisionale** (*decision boundary*) è rappresentato da una retta:

$$w_1 x_1 + w_2 x_2 + b = 0.$$

I parametri w_1, w_2 e b determinano quindi la posizione e l'orientamento del confine decisionale.

Il punteggio $f(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + b$ è continuo, ma la **classe predetta** è discreta. Una possibile regola di decisione binaria è

$$\hat{y} = \begin{cases} 1, & \text{se } f(\mathbf{x}) \geq 0, \\ 0, & \text{se } f(\mathbf{x}) < 0. \end{cases}$$

Il confine decisionale è dunque l'insieme dei punti per cui $f(\mathbf{x}) = 0$: in una dimensione coincide con un punto, in due con una retta, in tre con un piano e, più in generale, con un iperpiano. La scelta di assegnare il confine alla classe 1 è una convenzione; i valori 0 e 1 rappresentano *etichette*, non quantità da prevedere tramite regressione. La costruzione del classificatore e la scelta di una funzione di costo per apprenderne i parametri sono approfondite nella Sezione 12.

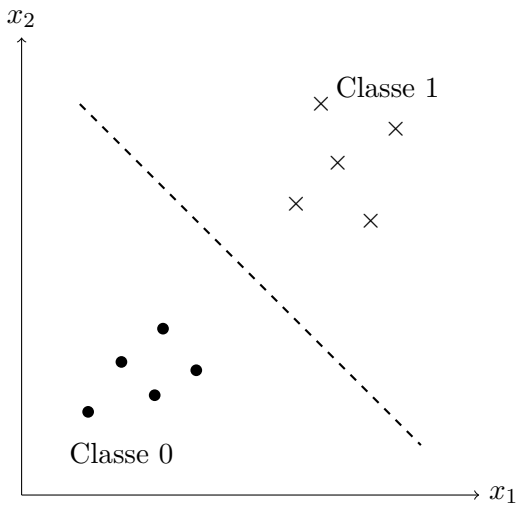


Figura 1: Esempio di classificazione lineare binaria in uno spazio delle caratteristiche bidimensionale.

Capitolo 5

Addestramento come problema di ottimizzazione

L'addestramento di un modello può essere interpretato come un **problema di ottimizzazione**.

Occorre infatti trovare i valori dei parametri $\mathbf{w}$ che permettono al modello di ottenere il comportamento desiderato.

A questo scopo viene definita una funzione che associa a ogni possibile configurazione dei parametri un valore numerico.

Indichiamo genericamente questa funzione con

$$g(\mathbf{w}).$$

Nel contesto del Machine Learning questa funzione può rappresentare, ad esempio, un costo o una quantità legata all'errore del modello.

Per vedere concretamente il collegamento, consideriamo un esempio puramente illustrativo: un modello molto semplice ha la forma

$$\hat{y} = f_w(x) = wx.$$

Se un esempio di addestramento ha $x = 2$ e output corretto $y = 6$, una possibile funzione di costo è il quadrato dell'errore:

$$g(w) = (f_w(2) - 6)^2 = (2w - 6)^2.$$

Il valore $w = 3$ annulla l'errore su questo esempio, mentre altri valori producono un costo maggiore. In un problema reale l'obiettivo si costruisce normalmente su un insieme di esempi, ma il principio non cambia: **addestrare significa scegliere i parametri che ottimizzano un criterio ricavato dai dati**. L'esempio mostra anche la differenza tra l'input x, il parametro w e la funzione di costo g.

L'obiettivo consiste quindi nel trovare una configurazione ottimale $\mathbf{w}^\star$.

Nel caso di un problema di minimizzazione:

$$\mathbf{w}^\star \in \arg\min_{\mathbf{w}} g(\mathbf{w}).$$

Il simbolo $\star$ indica quindi una soluzione ottimale.

5.1 Minimi e massimi

Data una funzione

$$g : \mathbb{R}^n \to \mathbb{R},$$

un punto $\mathbf{w}^\star$ è un **minimo globale** se

$$g(\mathbf{w}^\star) \leq g(\mathbf{w}) \qquad \forall \mathbf{w}.$$

Ciò significa che non esiste nessun altro punto del dominio in cui la funzione assuma un valore inferiore.

Analogamente, $\mathbf{w}^\star$ è un **massimo globale** se

$$g(\mathbf{w}^\star) \geq g(\mathbf{w}) \qquad \forall \mathbf{w}.$$

Un punto è invece un **minimo locale** quando il suo valore è minimo solamente rispetto ai punti sufficientemente vicini.

Formalmente, esiste un intorno di $\mathbf{w}^\star$ tale che

$$g(\mathbf{w}^\star) \leq g(\mathbf{w})$$

per tutti i punti $\mathbf{w}$ appartenenti a tale intorno.

La differenza fondamentale è quindi che:

- un minimo **globale** viene confrontato con tutti i punti possibili;
- un minimo **locale** viene confrontato solamente con i punti appartenenti a una regione circostante.

5.2 Trasformazione di un massimo in un minimo

Un problema di massimizzazione può essere trasformato in un problema di minimizzazione cambiando il segno della funzione.

Infatti:

$$\arg\max_{\mathbf{w}} g(\mathbf{w}) = \arg\min_{\mathbf{w}} [-g(\mathbf{w})].$$

Per questo motivo molti algoritmi di ottimizzazione possono essere formulati direttamente come problemi di minimizzazione.

5.3 Condizioni di ottimalità di ordine zero

Le definizioni precedenti confrontano direttamente i valori della funzione:

$$g(\mathbf{w}).$$

Per questo motivo vengono dette condizioni di ottimalità di **ordine zero**: non utilizzano informazioni sulle derivate della funzione, ma solamente i suoi valori.

5.4 Esempi

Consideriamo la funzione

$$g(w) = w^2 + 0.2.$$

Il suo minimo globale si trova in

$$w^\star = 0,$$

poiché

$$g(0) = 0.2$$

è il valore minimo assunto dalla funzione.

Passando a due variabili:

$$g(w_1, w_2) = w_1^2 + w_2^2 + 0.2,$$

il vettore ottimale è

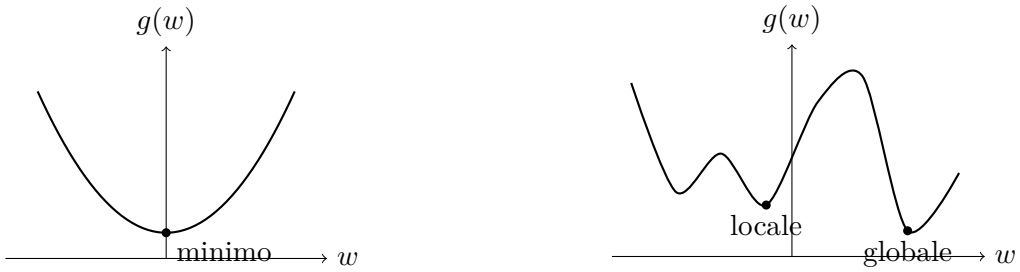
$$\mathbf{w}^\star = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

Capitolo 6

Convessità e difficoltà dell'ottimizzazione

La struttura della funzione da ottimizzare ha un'importanza fondamentale.

Una funzione può presentare diversi minimi e massimi locali. In questo caso individuare il minimo globale può risultare difficile.



Funzione convessa | Funzione non convessa

Figura 2: Confronto schematico tra una funzione convessa e una funzione non convessa.

Per una funzione convessa vale una proprietà particolarmente importante:

> Per una funzione convessa su un dominio convesso, ogni minimo locale è anche globale.

L'ottimizzazione risulta quindi generalmente più semplice.

In una funzione non convessa possono invece esistere diversi minimi locali. Un algoritmo potrebbe quindi raggiungere un minimo locale senza individuare il minimo globale.

Capitolo 7

Ottimizzazione globale

Una possibile strategia per cercare il minimo globale consiste nel **campionare** lo spazio dei parametri, calcolando la funzione in diversi punti e scegliendo quello che produce il valore migliore.

I punti possono essere scelti, ad esempio, mediante **campionamento uniforme** oppure **campionamento casuale**. Entrambe le strategie possono essere interpretate come approssimazioni della condizione di ottimalità di ordine zero, perché confrontano direttamente i valori assunti dalla funzione.

Aumentando la densità del campionamento si esplora lo spazio con una risoluzione più fine, senza però garantire di individuare esattamente il minimo globale.

Questo comporta però anche un aumento del costo computazionale.

In una sola dimensione il procedimento può essere relativamente semplice: si scelgono diversi valori di w e si valuta

$$g(w)$$

per ciascuno di essi.

Il problema diventa rapidamente più difficile quando aumenta il numero delle variabili.

7.1 Maledizione della dimensionalità

Supponiamo di utilizzare m punti di campionamento per ciascuna dimensione.

Con una sola variabile sono necessari:

$$m$$

punti.

Con due variabili diventano:

$$m^2.$$

Con tre variabili:

$$m^3.$$

In generale, con d dimensioni:

$$m^d.$$

Ad esempio, utilizzando solamente tre punti per ogni dimensione:

$$3^1 = 3, \qquad 3^2 = 9, \qquad 3^3 = 27.$$

Il numero di punti cresce quindi esponenzialmente con il numero delle dimensioni.

Questo fenomeno prende il nome di **maledizione della dimensionalità** (*curse of dimensionality*).

Dimensioni	Punti con 3 valori per dimensione
1	3
2	9
3	27
10	59049

L'esempio mostra perché un'esplorazione esaustiva dello spazio dei parametri diventa rapidamente impraticabile.

7.2 Campionamento casuale

Un'alternativa a una griglia regolare consiste nel valutare la funzione in punti scelti casualmente nello spazio dei parametri.

Il campionamento casuale permette di esplorare lo spazio senza dover costruire necessariamente una griglia completa.

Non garantisce però, da solo, di trovare il minimo globale.

Capitolo 8

Ottimizzazione locale

Poiché una ricerca globale del minimo può diventare rapidamente troppo costosa dal punto di vista computazionale, nella pratica si utilizzano spesso strategie di **ottimizzazione locale**.

L'idea generale consiste nel partire da un punto iniziale e migliorarlo progressivamente attraverso una sequenza di passi.

Nel contesto dell'addestramento di un modello, il punto corrente è rappresentato da un vettore di parametri:

$$\mathbf{w}^{(0)}.$$

Questo vettore contiene quindi i parametri, o pesi, del modello in un determinato istante dell'ottimizzazione.

8.1 Idea generale dell'ottimizzazione locale

Un algoritmo di ottimizzazione locale segue in generale il seguente schema:

1. si sceglie un punto iniziale $\mathbf{w}^{(0)}$;
2. si individua una direzione di discesa;
3. si effettua un passo in tale direzione;
4. si ottiene un nuovo punto $\mathbf{w}^{(1)}$;
5. il procedimento viene ripetuto in modo iterativo.

Se il nuovo punto produce un valore minore della funzione obiettivo, lo spostamento è considerato favorevole e l'algoritmo prosegue da lì.

In termini generali, al passo k si considera una direzione

$$\mathbf{d}^{(k-1)},$$

e si aggiorna il punto secondo la relazione:

$$\mathbf{w}^{(k)} = \mathbf{w}^{(k-1)} + \alpha \mathbf{d}^{(k-1)}.$$

Qui:

- $\mathbf{w}^{(k-1)}$ è il punto corrente;
- $\mathbf{d}^{(k-1)}$ è la direzione scelta al passo precedente;
- α è un valore scalare che controlla la lunghezza del passo.

Il parametro α viene chiamato **learning rate** (*tasso di apprendimento*).

8.2 Lunghezza del passo

La distanza percorsa durante il passo k può essere espressa mediante la norma euclidea della differenza tra due iterazioni successive:

$$\left\| \mathbf{w}^{(k)} - \mathbf{w}^{(k-1)} \right\|_2.$$

Sostituendo la formula di aggiornamento si ottiene:

$$\left\|\mathbf{w}^{(k)} - \mathbf{w}^{(k-1)}\right\|_2 = \left\|\left(\mathbf{w}^{(k-1)} + \alpha\mathbf{d}^{(k-1)}\right) - \mathbf{w}^{(k-1)}\right\|_2 = \left\|\alpha\mathbf{d}^{(k-1)}\right\|_2 .$$

Per omogeneità della norma:

$$\left\|\alpha\mathbf{d}^{(k-1)}\right\|_2 = |\alpha| \left\|\mathbf{d}^{(k-1)}\right\|_2 .$$

Se $\alpha > 0$, come normalmente avviene nei problemi di minimizzazione, si può scrivere:

$$\left\|\mathbf{w}^{(k)} - \mathbf{w}^{(k-1)}\right\|_2 = \alpha \left\|\mathbf{d}^{(k-1)}\right\|_2 .$$

Questa relazione mostra che la lunghezza del passo dipende sia dalla direzione scelta sia dal learning rate.

8.3 Norme vettoriali

Per misurare la grandezza di un vettore si possono utilizzare diverse norme.

Sia

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} .$$

Le principali norme introdotte sono:

- **norma** L_1:
$$\|\mathbf{x}\|_1 = \sum_{i=1}^{n} |x_i|;$$
- **norma** L_2:
$$\|\mathbf{x}\|_2 = \sqrt{\sum_{i=1}^{n} x_i^2};$$
- **norma** L_∞:
$$\|\mathbf{x}\|_\infty = \max_{1 \le i \le n} |x_i|.$$

Nel contesto dell'ottimizzazione locale, la norma L_2 è particolarmente naturale perché misura la distanza euclidea tra due punti nello spazio dei parametri.

8.4 Normalizzazione della direzione

Spesso conviene normalizzare la direzione di discesa imponendo:

$$\left\|\mathbf{d}^{(k-1)}\right\|_2 = 1.$$

In questo caso la lunghezza del passo diventa:

$$\left\|\mathbf{w}^{(k)} - \mathbf{w}^{(k-1)}\right\|_2 = \alpha.$$

Quindi, se la direzione viene normalizzata, la lunghezza del passo dipende solamente dal learning rate.

Questo chiarisce il ruolo di α: esso controlla direttamente l'ampiezza dello spostamento da un'iterazione alla successiva.

8.5 Learning rate e iperparametri

Il learning rate è un esempio di **iperparametro**.

Gli iperparametri sono parametri che non rappresentano i pesi del modello da apprendere, ma configurano il comportamento dell'algoritmo di addestramento o del modello stesso.

È quindi importante distinguere:

- i **parametri del modello**, che vogliamo determinare durante l'addestramento;
- gli **iperparametri**, che devono essere scelti o configurati per guidare il processo di apprendimento.

La scelta degli iperparametri influenza fortemente il risultato finale.

In particolare, il learning rate può essere:

- **fisso**, se mantiene sempre lo stesso valore;
- **variabile**, se cambia durante l'ottimizzazione secondo una politica di *scheduling*.

Esistono inoltre tecniche specifiche per la scelta o l'ottimizzazione degli iperparametri.

8.6 Significato del learning rate

Il learning rate regola l'entità degli spostamenti nell'ottimizzazione locale. Un valore molto piccolo può rallentare il procedimento; un valore troppo grande può portare a salti eccessivi e a mancati miglioramenti.

L'effetto dipende dalla funzione, dalla direzione scelta e dalla regola di aggiornamento. Per questo, i casi di convergenza lenta, oscillazione e divergenza saranno analizzati in modo concreto quando introdurremo la *gradient descent*.

8.7 Regola del passo decrescente

In alternativa a un learning rate costante, è possibile ridurre progressivamente la lunghezza del passo durante l'ottimizzazione.

Questa strategia prende il nome di **regola del passo decrescente** (*diminishing steplength rule*).

Un esempio è:

$$\alpha_k = \frac{1}{k}.$$

Se la direzione è normalizzata in modo che

$$\|\mathbf{d}^{(k-1)}\|_2 = 1,$$

allora la distanza percorsa al passo k è:

$$\left\|\mathbf{w}^{(k)} - \mathbf{w}^{(k-1)}\right\|_2 = \alpha_k = \frac{1}{k}.$$

I singoli passi diventano quindi progressivamente più piccoli.

La distanza complessivamente percorsa dopo K iterazioni è:

$$\sum_{k=1}^{K} \left\|\mathbf{w}^{(k)} - \mathbf{w}^{(k-1)}\right\|_2 = \sum_{k=1}^{K} \frac{1}{k}.$$

La serie armonica diverge: benché ciascun passo diventi progressivamente più piccolo, la somma delle lunghezze dei passi non converge a una distanza totale finita.

8.8 Random search

Una possibile strategia di ottimizzazione locale è il **random search**.

L'idea è la seguente: dato il punto corrente

$$\mathbf{w}^{(k-1)},$$

si campiona un insieme finito di direzioni casuali e si valuta quale di esse produca il miglior valore della funzione obiettivo tra i candidati.

Siano quindi generate P direzioni candidate:

$$\mathbf{d}^1, \mathbf{d}^2, \ldots, \mathbf{d}^P.$$

Ciascuna direzione definisce un punto candidato:

$$\mathbf{w}^{(k-1)} + \mathbf{d}^p, \qquad p = 1, \ldots, P.$$

Si seleziona quindi l'indice

$$s = \arg\min_{p=1,\ldots,P} g\Big(\mathbf{w}^{(k-1)} + \mathbf{d}^p\Big).$$

L'indice s individua la direzione che, tra quelle campionate, produce il valore minore della funzione obiettivo.

Se tale valore è effettivamente migliore del punto corrente, cioè se

$$g\Big(\mathbf{w}^{(k-1)} + \mathbf{d}^s\Big) < g\Big(\mathbf{w}^{(k-1)}\Big),$$

allora si aggiorna il punto:

$$\mathbf{w}^{(k)} = \mathbf{w}^{(k-1)} + \mathbf{d}^s.$$

In caso contrario, si può interrompere il procedimento oppure generare un nuovo insieme di direzioni candidate.

8.9 Esempio di random search

Consideriamo la funzione quadratica:

$$g(w_1, w_2) = w_1^2 + w_2^2 + 2.$$

Questa funzione ha minimo globale nel punto

$$\mathbf{w}^\star = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

Supponiamo di partire da un punto iniziale, ad esempio

$$\mathbf{w}^{(0)} = \begin{bmatrix} 3 \\ 4 \end{bmatrix}.$$

A ogni iterazione si campionano più direzioni casuali a partire dal punto corrente, si valuta la funzione nei punti candidati e si sceglie la direzione che produce la riduzione maggiore del costo.

Ripetendo il procedimento, si possono ottenere punti con costo progressivamente minore e, nei casi favorevoli, avvicinarsi al minimo. La convergenza non è comunque garantita dalla sola casualità delle direzioni.

8.10 Limiti del random search e dimensionalità

Nel random search le direzioni candidate vengono campionate casualmente nello spazio delle possibili direzioni.

Il metodo può funzionare bene in spazi di piccola dimensione, ma diventa progressivamente meno efficace all'aumentare del numero dei parametri.

Il problema è collegato nuovamente alla **maledizione della dimensionalità**: per molte funzioni diventa esponenzialmente più difficile trovare casualmente una direzione di discesa quando aumenta la dimensione dello spazio di input.

Un esempio significativo è la funzione quadratica

$$g(\mathbf{w}) = \mathbf{w}^T\mathbf{w} + 2.$$

Nell'esempio considerato dal docente, con dimensionalità pari a

$$N = 30,$$

la probabilità di campionare casualmente una direzione di discesa è indicata come inferiore all'1%. Tale valore si riferisce alle condizioni dell'esempio: non è una probabilità universale determinata dalla sola funzione quadratica e dal numero di dimensioni.

Questo permette di comprendere perché il random search possa diventare molto inefficiente in alta dimensione: la singola valutazione della funzione non diventa necessariamente esponenzialmente più costosa, ma può essere necessario esplorare un numero molto elevato di direzioni prima di trovarne una favorevole.

8.11 Ricerca per coordinate

Un modo per evitare di campionare direzioni completamente casuali consiste nello sfruttare direttamente gli assi dello spazio dei parametri.

La **ricerca per coordinate** (*coordinate search*) considera direzioni allineate con le coordinate.

Indichiamo con

$$\mathbf{e}_1, \mathbf{e}_2, \ldots, \mathbf{e}_n$$

i vettori della base canonica.

Per ogni coordinata i vengono considerate le due possibili direzioni:

$$+\mathbf{e}_i \qquad \text{e} \qquad -\mathbf{e}_i.$$

A partire dal punto corrente $\mathbf{w}^{(k)}$, i punti candidati possono quindi essere scritti come:

$$\mathbf{w}^{(k)} + \alpha\mathbf{e}_i$$

e

$$\mathbf{w}^{(k)} - \alpha\mathbf{e}_i, \qquad i = 1, \ldots, n.$$

In uno spazio di dimensione n vengono quindi considerate $2n$ **direzioni**.

Il numero di direzioni cresce linearmente con la dimensionalità:

$$2n = O(n).$$

Questo comportamento è molto più controllabile rispetto alla ricerca casuale di un insieme sufficientemente grande di direzioni in uno spazio ad alta dimensionalità.

8.12 Discesa per coordinate

Una variante è la **discesa per coordinate** (*coordinate descent*).

Invece di considerare simultaneamente tutte le coordinate, l'algoritmo ne modifica solamente una, oppure un sottoinsieme, alla volta.

Ad esempio, in due dimensioni si potrebbe aggiornare prima w_1, successivamente w_2, poi nuovamente w_1 e così via.

Indicando con i_k la coordinata scelta all'iterazione k, un aggiornamento può essere rappresentato schematicamente come:

$$\mathbf{w}^{(k)} = \mathbf{w}^{(k-1)} + \alpha s_k \mathbf{e}_{i_k},$$

dove s_k determina il verso dello spostamento lungo la coordinata considerata.

La scelta delle coordinate può essere effettuata secondo una politica **ciclica**, ad esempio:

$$1, 2, \ldots, n, 1, 2, \ldots, n, \ldots$$

oppure considerando piccoli sottoinsiemi di coordinate alla volta.

Il principale vantaggio è che ogni singolo aggiornamento coinvolge solo una parte dello spazio dei parametri.

Il prezzo da pagare è che la direzione geometricamente migliore potrebbe non essere allineata con gli assi: il percorso verso il minimo può quindi risultare più lungo o assumere un caratteristico andamento a "gradini".

Costo delle due strategie

La coordinate search che prova i due versi di ciascuno dei n assi considera $2n$ direzioni candidate per una scansione completa.

La coordinate descent aggiorna invece una coordinata alla volta; per decidere quanto e dove muoversi può usare una semplice valutazione, una ricerca unidimensionale oppure anche informazioni derivate.

Quindi **numero di coordinate aggiornate**, **numero di valutazioni della funzione** e **costo computazionale** non sono la stessa cosa. In una semplice scansione il numero di direzioni esaminate può crescere linearmente con n (n o $2n$, a seconda dello schema), ma il costo complessivo dipende anche dalle operazioni necessarie per scegliere il passo e calcolare il valore della funzione.

8.13 History plot della funzione costo

Un modo utile per osservare l'andamento dell'ottimizzazione consiste nel rappresentare il valore della funzione obiettivo a ogni iterazione.

Si considera quindi il grafico di:

$$g\Big(\mathbf{w}^{(k)}\Big)$$

in funzione dello step k.

Questo grafico viene spesso chiamato **cost function history plot** oppure semplicemente **history plot**.

Esso permette di osservare la "storia" dell'ottimizzazione: se il costo decresce in modo regolare, se si stabilizza, se oscilla oppure se smette di migliorare.

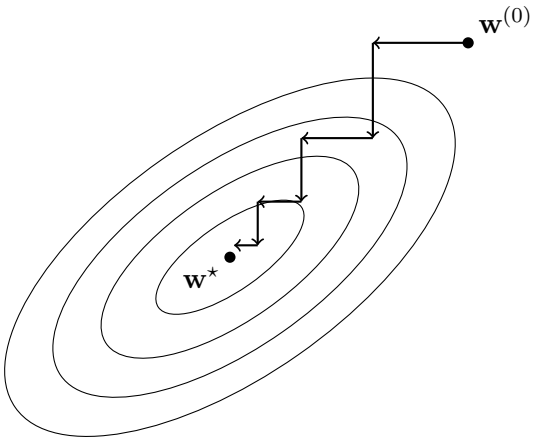


Figura 3: Esempio schematico di discesa per coordinate. Gli aggiornamenti avvengono lungo gli assi e possono produrre un percorso a gradini verso il minimo.

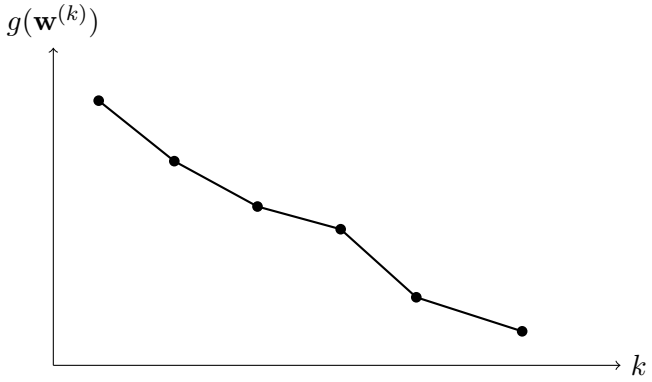


Figura 4: Esempio schematico di history plot della funzione costo.

8.14 Ottimizzazione locale e funzioni non convesse

Nel caso di funzioni non convesse, l'ottimizzazione locale può convergere a minimi locali diversi a seconda del punto iniziale.

Per questo motivo, nella minimizzazione di funzioni non convesse, può essere utile eseguire **più run indipendenti** dell'algoritmo, ciascuna con un punto iniziale differente.

In questo modo aumenta la probabilità di trovare soluzioni migliori.

In sintesi:

- per funzioni **convesse**, un minimo locale è anche globale;
- per funzioni **non convesse**, l'algoritmo può fermarsi in un minimo locale non globale;
- per questo motivo può essere utile riavviare l'ottimizzazione più volte.

Capitolo 9

Ottimizzazione del primo ordine e discesa del gradiente

Il random search e le varianti di ricerca per coordinate basate soltanto sui valori della funzione sono esempi di tecniche di **ordine zero**: per scegliere come muoversi utilizzano valutazioni della funzione obiettivo

$$g(\mathbf{w}).$$

Le tecniche di **primo ordine** sfruttano invece anche l'informazione contenuta nelle derivate della funzione. In più dimensioni questa informazione è raccolta nel **gradiente**. La distinzione riguarda le informazioni utilizzate, non necessariamente il fatto di aggiornare una o più coordinate.

9.1 Gradiente e punti stazionari

Per una funzione differenziabile

$$g : \mathbb{R}^N \to \mathbb{R},$$

il gradiente è il vettore delle derivate parziali:

$$\nabla g(\mathbf{w}) = \begin{bmatrix} \dfrac{\partial g}{\partial w_1} \\ \dfrac{\partial g}{\partial w_2} \\ \vdots \\ \dfrac{\partial g}{\partial w_N} \end{bmatrix} \in \mathbb{R}^{N \times 1}.$$

Il gradiente descrive come varia localmente la funzione rispetto alle diverse coordinate dello spazio dei parametri.

Un punto $\mathbf{w}^\star$ è detto **stazionario** quando

$$\nabla g(\mathbf{w}^\star) = \mathbf{0}_{N \times 1}.$$

In una sola dimensione questa condizione coincide con

$$g'(w^\star) = 0.$$

Geometricamente, in un punto stazionario la retta tangente, oppure il piano o iperpiano tangente in più dimensioni, è localmente piatto.

La condizione

$$\nabla g(\mathbf{w}^\star) = \mathbf{0}$$

è una condizione necessaria per i minimi e massimi locali interni di una funzione differenziabile, ma in generale non è sufficiente per stabilire il tipo di punto. Un punto stazionario può infatti essere:

- un minimo locale;
- un massimo locale;
- un punto di sella, in più dimensioni;
- un punto stazionario non estremo, come un flesso stazionario in una dimensione.

Per le funzioni convesse, invece, un punto stazionario corrisponde a un minimo globale. La risoluzione simultanea del sistema

$$\nabla g(\mathbf{w}) = \mathbf{0}$$

può diventare difficile quando il numero delle variabili è elevato. In alcuni casi le equazioni possono essere affrontate una alla volta, procedendo sequenzialmente rispetto alle coordinate: questa idea presenta un collegamento diretto con i metodi di *coordinate descent*.

9.2 Approssimazione lineare tramite Taylor

Per comprendere come utilizzare il gradiente per scegliere una direzione di movimento, consideriamo dapprima un generico iperpiano affine:

$$h(\mathbf{w}) = a + b_1 w_1 + b_2 w_2 + \cdots + b_N w_N.$$

Introducendo il vettore dei coefficienti

$$\mathbf{b} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_N \end{bmatrix},$$

la stessa espressione può essere scritta in forma compatta come

$$h(\mathbf{w}) = a + \mathbf{b}^T \mathbf{w}.$$

Una funzione differenziabile a più variabili può essere approssimata localmente, in un intorno di un punto $\mathbf{w}^{(0)}$, mediante la serie di Taylor arrestata al primo ordine:

$$h(\mathbf{w}) = g(\mathbf{w}^{(0)}) + \nabla g(\mathbf{w}^{(0)})^T \left(\mathbf{w} - \mathbf{w}^{(0)}\right).$$

Sviluppando l'espressione:

$$h(\mathbf{w}) = \underbrace{g(\mathbf{w}^{(0)}) - \nabla g(\mathbf{w}^{(0)})^T \mathbf{w}^{(0)}}_{a} + \underbrace{\nabla g(\mathbf{w}^{(0)})^T}_{\mathbf{b}^T} \mathbf{w}.$$

Confrontando questa forma con

$$h(\mathbf{w}) = a + \mathbf{b}^T \mathbf{w},$$

si osserva quindi che

$$\mathbf{b} = \nabla g(\mathbf{w}^{(0)}).$$

L'iperpiano h è tangente a g nel punto $\mathbf{w}^{(0)}$ e ne descrive il comportamento locale. Questo collegamento permette di trasformare un problema geometrico sull'iperpiano in un criterio per scegliere come muoversi sulla funzione originale.

Per una funzione

$$g : \mathbb{R}^N \to \mathbb{R},$$

l'input possiede N coordinate, mentre il valore della funzione aggiunge una coordinata ulteriore: il grafico della funzione vive quindi in uno spazio $\mathbb{R}^{N+1}$.

9.3 Direzione di massima salita e massima discesa

Consideriamo l'iperpiano

$$h(\mathbf{w}) = a + \mathbf{b}^T\mathbf{w}$$

nel punto $\mathbf{w}^{(0)}$ e cerchiamo una direzione unitaria $\mathbf{d}$, con

$$\|\mathbf{d}\|_2 = 1.$$

Spostandoci di una unità nella direzione $\mathbf{d}$ otteniamo:

$$h(\mathbf{w}^{(0)} + \mathbf{d}) = a + \mathbf{b}^T\mathbf{w}^{(0)} + \mathbf{b}^T\mathbf{d}.$$

I primi due termini non dipendono da $\mathbf{d}$. Per trovare la direzione che produce la variazione maggiore è quindi sufficiente studiare

$$\mathbf{b}^T\mathbf{d}.$$

Per la definizione di prodotto scalare:

$$\mathbf{b}^T\mathbf{d} = \|\mathbf{b}\|_2 \, \|\mathbf{d}\|_2 \cos\theta.$$

Poiché $\|\mathbf{d}\|_2 = 1$ e $\|\mathbf{b}\|_2$ non dipende dalla direzione scelta, tutto dipende dal valore di $\cos\theta$.

La **massima salita** si ottiene per

$$\theta = 0, \qquad \cos\theta = 1,$$

e quindi

$$\mathbf{d}_{\text{asc}} = \frac{\mathbf{b}}{\|\mathbf{b}\|_2}.$$

La **massima discesa** si ottiene invece per

$$\theta = \pi, \qquad \cos\theta = -1,$$

e quindi

$$\mathbf{d}_{\text{desc}} = -\frac{\mathbf{b}}{\|\mathbf{b}\|_2}.$$

Poiché nell'approssimazione di Taylor vale

$$\mathbf{b} = \nabla g(\mathbf{w}^{(0)}),$$

se il gradiente non è nullo, questo individua la direzione di massima salita locale e il suo opposto quella di massima discesa locale, **tra tutte le direzioni di norma euclidea unitaria**:

$$\mathbf{d}_{\text{asc}} = \frac{\nabla g(\mathbf{w}^{(0)})}{\|\nabla g(\mathbf{w}^{(0)})\|_2}, \qquad \mathbf{d}_{\text{desc}} = -\frac{\nabla g(\mathbf{w}^{(0)})}{\|\nabla g(\mathbf{w}^{(0)})\|_2}.$$

Se invece $\nabla g(\mathbf{w}^{(0)}) = \mathbf{0}$, le formule normalizzate non sono definite: ci troviamo in un punto stazionario e l'approssimazione del primo ordine non distingue una direzione preferita.

Questa è l'idea fondamentale che conduce alla discesa del gradiente.

9.4 Gradiente e curve di livello

Una proprietà del **gradiente in generale**, non soltanto dell'algoritmo di discesa del gradiente, è che per una funzione differenziabile e in un punto con gradiente non nullo esso è perpendicolare alla curva o superficie di livello che passa per quel punto:

$$g(\mathbf{w}) = g(\mathbf{w}^{(0)}).$$

La direzione di discesa $-\nabla g(\mathbf{w}^{(0)})$ è quindi anch'essa perpendicolare alla curva di livello, ma orientata verso valori inferiori della funzione.

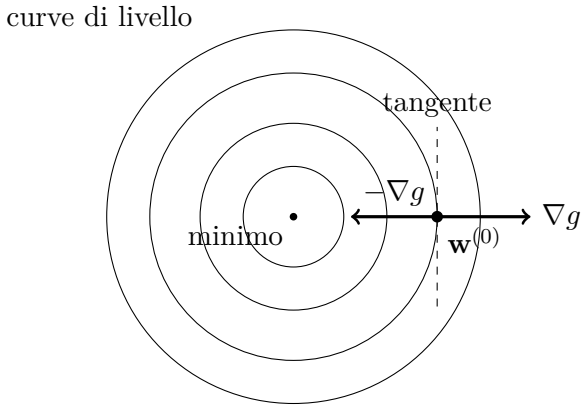


Figura 5: Per $g(w_1, w_2) = w_1^2 + w_2^2$, il gradiente non nullo è perpendicolare alla curva di livello nel punto evidenziato. Il suo opposto indica il verso di discesa locale.

Questa proprietà aiuta anche a interpretare i percorsi dell'ottimizzazione: in una valle stretta, la normale alle curve di livello può cambiare orientamento rapidamente, contribuendo al comportamento a zig-zag che verrà discusso più avanti.

9.5 Algoritmo di discesa del gradiente

La **discesa del gradiente** (*gradient descent*) utilizza il gradiente negativo come direzione di discesa quando è non nullo. Infatti, per un gradiente non nullo, la derivata direzionale nella direzione $-\nabla g(\mathbf{w})$ è negativa:

$$\nabla g(\mathbf{w})^T[-\nabla g(\mathbf{w})] = -\|\nabla g(\mathbf{w})\|_2^2 < 0.$$

Questo significa che **per spostamenti sufficientemente piccoli** la funzione diminuisce. La scelta di un passo finito, invece, non garantisce automaticamente una riduzione del costo: qui entra in gioco il learning rate.

Partendo dal generico schema di ottimizzazione locale

$$\mathbf{w}^{(k)} = \mathbf{w}^{(k-1)} + \alpha \mathbf{d}^{(k-1)},$$

si sceglie

$$\mathbf{d}^{(k-1)} = -\nabla g(\mathbf{w}^{(k-1)}).$$

Si ottiene quindi la regola di aggiornamento:

$$\boxed{\mathbf{w}^{(k)} = \mathbf{w}^{(k-1)} - \alpha \nabla g(\mathbf{w}^{(k-1)})}$$

con k indice dell'iterazione e $\alpha > 0$ learning rate.

È importante osservare una differenza rispetto al caso in cui la direzione sia normalizzata. Nella gradient descent standard si utilizza il gradiente negativo senza imporre

$$\|\nabla g(\mathbf{w})\|_2 = 1.$$

Di conseguenza, la lunghezza del passo dipende sia da α sia dalla magnitudine del gradiente:

$$\left\|\mathbf{w}^{(k)} - \mathbf{w}^{(k-1)}\right\|_2 = \alpha \left\|\nabla g(\mathbf{w}^{(k-1)})\right\|_2 .$$

Esempio numerico di aggiornamento

Consideriamo la funzione quadratica a due parametri già utilizzata come esempio:

$$g(w_1, w_2) = w_1^2 + w_2^2 + 2, \qquad \nabla g(\mathbf{w}) = \begin{bmatrix} 2w_1 \\ 2w_2 \end{bmatrix}.$$

Partiamo da $\mathbf{w}^{(0)} = (3, 4)^T$ con $\alpha = 0.1$. Il gradiente vale $(6, 8)^T$ e il primo aggiornamento è

$$\mathbf{w}^{(1)} = \begin{bmatrix} 3 \\ 4 \end{bmatrix} - 0.1 \begin{bmatrix} 6 \\ 8 \end{bmatrix} = \begin{bmatrix} 2.4 \\ 3.2 \end{bmatrix}.$$

Il costo diminuisce da $g(3, 4) = 27$ a $g(2.4, 3.2) = 18$. L'esempio mostra, con numeri concreti, come la regola iterativa trasformi i parametri e come valutarne l'effetto sulla funzione obiettivo.

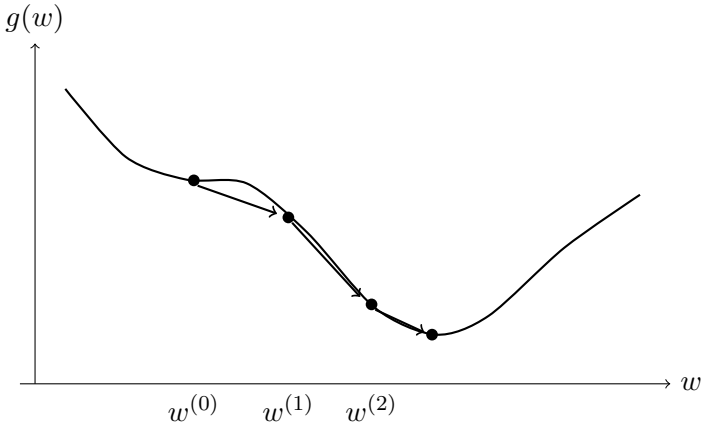


Figura 6: Schema qualitativo della discesa del gradiente su una funzione a singolo input. Ogni aggiornamento utilizza una direzione locale di discesa.

Per una funzione convessa, la discesa locale conduce verso il minimo globale, purché il learning rate permetta una convergenza corretta. Per una funzione non convessa, invece, il

risultato può dipendere dal punto di inizializzazione e l'algoritmo può raggiungere minimi locali differenti.

9.6 Scelta del learning rate nella gradient descent

Il learning rate α controlla l'ampiezza dell'aggiornamento nella direzione del gradiente negativo. Come abbiamo visto in generale, la sua scelta può influenzare sia la velocità sia la stabilità dell'ottimizzazione. Ora possiamo illustrarlo su un algoritmo preciso.

Una scelta comune consiste nell'utilizzare un valore fisso dell'ordine di

$$\alpha = 10^{\gamma}, \qquad \gamma \in \mathbb{Z}, \qquad \gamma < 0,$$

ad esempio 10^{-1}, 10^{-2} o 10^{-3}. Non è però necessario che il learning rate sia una potenza esatta di dieci: ciò che conta è la scala del passo.

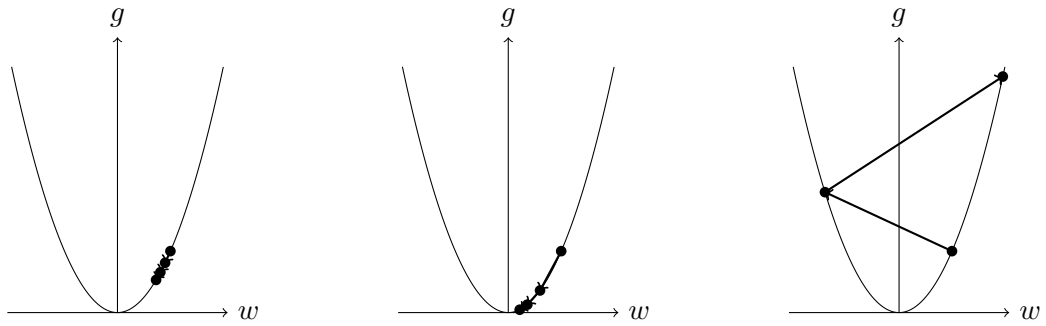
In alternativa si può utilizzare una regola decrescente, come

$$\alpha_k = \frac{1}{k}.$$

Per esempio, su $g(w) = w^2$ il gradiente è $2w$, quindi gli aggiornamenti soddisfano $w^{(k+1)} = (1 - 2\alpha)w^{(k)}$. Questa relazione permette di verificare esattamente l'effetto del learning rate (Figura 7).

Gli effetti della scelta di α possono essere molto diversi:

- un valore troppo piccolo produce passi corti e quindi un progresso molto lento;
- un valore adeguato permette una discesa regolare verso una regione di minimo;
- un valore troppo grande può causare oscillazioni, salti da un lato all'altro della valle oppure addirittura un aumento della funzione obiettivo e mancata convergenza.



$\alpha = 0.05$: lenta $\alpha = 0.20$: rapida $\alpha = 1.20$: diverge

Figura 7: Gradient descent su $g(w) = w^2$ a partire da $w^{(0)} = 2.5$. I punti sono iterazioni calcolate tramite $w^{(k+1)} = (1 - 2\alpha)w^{(k)}$. Passi piccoli rallentano, passi adeguati convergono, un valore eccessivo può oscillare amplificando l'errore.

L'**history plot** (Figura 4) permette di confrontare diverse esecuzioni. Se si considera soltanto la funzione obiettivo dell'addestramento, è utile osservare anche il valore minimo raggiunto nel corso delle iterazioni, non soltanto il valore finale. Ciò non significa però che l'iterazione con costo di training minimo sia automaticamente il modello migliore sui dati nuovi: la validazione serve proprio a controllare la generalizzazione.

9.7 Criteri di convergenza

In pratica è necessario stabilire quando interrompere la procedura iterativa. Il simbolo

$$\approx$$

indica che due quantità sono considerate **approssimativamente uguali** secondo una tolleranza prefissata.

Se le iterazioni convergono verso un punto stazionario e il gradiente è continuo, il gradiente tende a zero:

$$-\nabla g(\mathbf{w}^{(k)}) \approx \mathbf{0}_{N\times 1}.$$

Una possibile condizione di arresto è quindi

$$\left\|\nabla g(\mathbf{w}^{(k-1)})\right\|_2 < \varepsilon.$$

Questa condizione segnala la vicinanza a un punto stazionario, che non è necessariamente un minimo globale: può essere anche un massimo o un punto di sella.

Altri criteri considerano quanto cambiano i parametri o il valore della funzione tra due iterazioni consecutive:

$$\frac{1}{N}\left\|\mathbf{w}^{(k)} - \mathbf{w}^{(k-1)}\right\|_2 < \varepsilon,$$

oppure

$$\frac{1}{N}\left|g(\mathbf{w}^{(k)}) - g(\mathbf{w}^{(k-1)})\right| < \varepsilon.$$

Il valore $\varepsilon > 0$ è una tolleranza prestabilita. Queste sono **possibili condizioni di arresto alternative**, non prove automatiche di aver raggiunto un minimo globale. Ad esempio, un learning rate molto piccolo può produrre passi quasi nulli anche se il gradiente non è ancora piccolo.

Dal punto di vista pratico può essere inoltre fissato un numero massimo di iterazioni. Questa scelta dipende dalla funzione, dalle risorse computazionali disponibili e dal learning rate utilizzato.

9.8 Perché utilizzare il gradiente

Rispetto a una ricerca puramente casuale delle direzioni, il gradiente fornisce direttamente informazione locale sulla direzione in cui la funzione cresce più rapidamente. Di conseguenza, quando il gradiente è non nullo, il suo opposto fornisce una direzione locale valida di discesa.

Questo evita di dover campionare e confrontare un grande numero di direzioni candidate come nel random search, purché il gradiente sia calcolabile e non nullo. Per questo motivo la gradient descent non soffre della maledizione della dimensionalità nello stesso modo della ricerca casuale delle direzioni, anche se il costo di calcolare il gradiente può naturalmente aumentare con la dimensione del modello.

9.9 Limiti della discesa del gradiente

Il gradiente è un vettore caratterizzato da **direzione** e **magnitudine**. Entrambe queste componenti possono produrre comportamenti problematici.

9.9.1 Zig-zag

In alcune funzioni, in particolare in presenza di valli strette e allungate, la direzione del gradiente negativo può cambiare rapidamente da un lato all'altro della valle. Il percorso assume quindi un andamento a zig-zag e la convergenza lungo la direzione effettivamente utile diventa lenta.

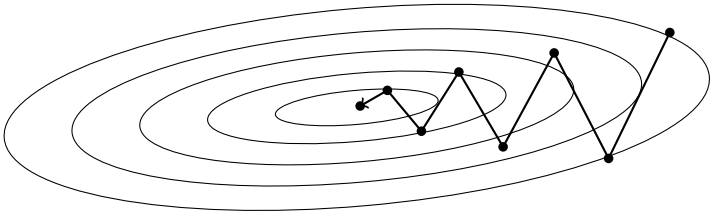


Figura 8: Esempio schematico di comportamento a zig-zag in una valle stretta e allungata.

9.9.2 Slow-crawling

In prossimità di un punto stazionario vale

$$\nabla g(\mathbf{w}) \approx \mathbf{0}.$$

Poiché nella gradient descent la distanza percorsa al passo k è

$$\left\|\mathbf{w}^{(k)} - \mathbf{w}^{(k-1)}\right\|_2 = \alpha \left\|\nabla g(\mathbf{w}^{(k-1)})\right\|_2,$$

quando la magnitudine del gradiente diventa molto piccola anche gli spostamenti diventano molto piccoli. Il progresso dell'algoritmo può quindi rallentare sensibilmente: questo comportamento prende il nome di **slow-crawling**.

Il fenomeno è particolarmente evidente in funzioni caratterizzate da **long narrow valleys**, cioè regioni lunghe e strette con zone relativamente piatte e curve di livello progressivamente quasi parallele. La Figura 9 illustra in una dimensione il progressivo accorciamento dei passi quando il gradiente si riduce.

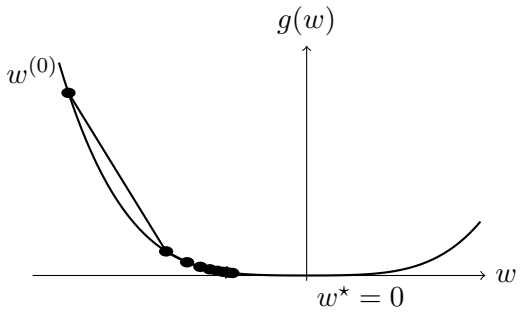


Figura 9: Esempio di slow-crawling su $g(w) = w^4 + 0.05w^2$: con learning rate costante $\alpha = 0.1$, la distanza tra iterazioni successive si riduce man mano che il gradiente diventa piccolo.

Capitolo 10

Ottimizzazione del secondo ordine

La discesa del gradiente utilizza informazioni di **primo ordine**: il gradiente indica come cambia localmente la funzione e permette di scegliere una direzione di discesa. Tuttavia, la sola pendenza non descrive quanto rapidamente questa cambi. Due funzioni possono avere la stessa derivata in un punto, ma comportarsi diversamente nelle sue vicinanze.

Le tecniche di **secondo ordine** sfruttano anche la **curvatura**, descritta dalle derivate seconde. Questa informazione permette di studiare con maggiore precisione la forma locale della funzione e di costruirne un'approssimazione quadratica. Il metodo di Newton costituisce un esempio di utilizzo di questa idea.

10.1 Derivate seconde e matrice Hessiana

Per una funzione di una variabile, la derivata prima $g'(w)$ descrive la pendenza locale, mentre la derivata seconda $g''(w)$ descrive come cambia tale pendenza. Quando $g''(w) > 0$, la funzione presenta localmente una curvatura rivolta verso l'alto; quando $g''(w) < 0$, la curvatura è rivolta verso il basso. Se la derivata seconda è non negativa su tutto un intervallo, la funzione è convessa in quell'intervallo; se è non positiva, è concava.

Per estendere lo stesso ragionamento a una funzione di N variabili

$$g : \mathbb{R}^N \longrightarrow \mathbb{R},$$

si introduce la **matrice Hessiana**, che raccoglie tutte le **derivate parziali seconde**:

$$\boxed{\nabla^2 g(\mathbf{w}) = \begin{bmatrix} \dfrac{\partial^2 g}{\partial w_1^2} & \dfrac{\partial^2 g}{\partial w_1 \partial w_2} & \cdots & \dfrac{\partial^2 g}{\partial w_1 \partial w_N} \\ \dfrac{\partial^2 g}{\partial w_2 \partial w_1} & \dfrac{\partial^2 g}{\partial w_2^2} & \cdots & \dfrac{\partial^2 g}{\partial w_2 \partial w_N} \\ \vdots & \vdots & \ddots & \vdots \\ \dfrac{\partial^2 g}{\partial w_N \partial w_1} & \dfrac{\partial^2 g}{\partial w_N \partial w_2} & \cdots & \dfrac{\partial^2 g}{\partial w_N^2} \end{bmatrix}}$$

In forma compatta, l'elemento di riga i e colonna j è

$$[\nabla^2 g(\mathbf{w})]_{ij} = \frac{\partial^2 g(\mathbf{w})}{\partial w_i \, \partial w_j}.$$

L'Hessiana è quindi una matrice $N \times N$: sulla diagonale contiene le derivate seconde rispetto a una singola coordinata, mentre fuori dalla diagonale compaiono le derivate miste, che descrivono come la variazione lungo una coordinata dipenda dalle altre. Se le derivate parziali seconde sono continue, le derivate miste coincidono a coppie e l'Hessiana è una **matrice simmetrica**:

$$\nabla^2 g(\mathbf{w}) = \left(\nabla^2 g(\mathbf{w})\right)^T.$$

Per interpretare la curvatura in più dimensioni è utile richiamare brevemente autovalori e autovettori.

10.2 Richiamo su autovalori e autovettori

Una matrice quadrata $A \in \mathbb{R}^{N\times N}$ rappresenta una trasformazione lineare. In generale, quando la si applica a un vettore, può cambiarne sia la lunghezza sia la direzione. Un **autovettore** $\mathbf{x} \neq \mathbf{0}$ è invece un vettore che la trasformazione lascia sulla stessa retta, modificandone soltanto il fattore di scala:

$$\boxed{A\mathbf{x} = \lambda\mathbf{x}.}$$

Il numero λ è l'**autovalore** associato all'autovettore $\mathbf{x}$. Se $\lambda > 0$ il verso è conservato, se $\lambda < 0$ è invertito, mentre se $\lambda = 0$ il vettore viene mandato nel vettore nullo. Per una matrice reale generica gli autovalori possono anche essere complessi; per una **matrice reale simmetrica**, come l'Hessiana nelle ipotesi precedenti, gli autovalori sono sempre reali.

Per esempio, considerando

$$A = \begin{bmatrix} 2 & 0 \\ 0 & -1 \end{bmatrix}, \qquad \mathbf{e}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \qquad \mathbf{e}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix},$$

si ottiene $A\mathbf{e}_1 = 2\mathbf{e}_1$ e $A\mathbf{e}_2 = -\mathbf{e}_2$. Le due direzioni coordinate sono quindi autovettori, rispettivamente di autovalori 2 e -1. Nell'analisi della curvatura, il segno degli autovalori dell'Hessiana permette di riconoscere se le diverse direzioni presentano una curvatura rivolta verso l'alto o verso il basso.

10.3 Convessità e condizioni di ottimalità del secondo ordine

Per una matrice Hessiana reale e simmetrica si utilizzano le seguenti definizioni:

- è **semidefinita positiva** se tutti gli autovalori sono non negativi;
- è **semidefinita negativa** se tutti gli autovalori sono non positivi;
- è **definita positiva** se tutti gli autovalori sono strettamente positivi;
- è **definita negativa** se tutti gli autovalori sono strettamente negativi;
- è **indefinita** se possiede autovalori sia positivi sia negativi.

Per una funzione due volte continuamente differenziabile su un **dominio convesso**, l'Hessiana semidefinita positiva in *ogni punto del dominio* caratterizza una funzione convessa; se è semidefinita negativa in ogni punto, la funzione è concava. Il segno dell'Hessiana in un solo punto, invece, fornisce informazioni sulla **curvatura locale**, ma non dimostra la convessità globale.

Questi strumenti permettono anche di classificare un **punto stazionario** $\mathbf{v}$, per il quale $\nabla g(\mathbf{v}) = \mathbf{0}$, mediante condizioni sufficienti di secondo ordine:

- se $\nabla^2 g(\mathbf{v})$ è definita positiva, allora $\mathbf{v}$ è un **minimo locale stretto**;
- se è definita negativa, allora $\mathbf{v}$ è un **massimo locale stretto**;
- se è indefinita, allora $\mathbf{v}$ è un **punto di sella**.

Queste conclusioni sono locali: un minimo locale è anche globale *solo sotto opportune ipotesi aggiuntive*, ad esempio se la funzione è convessa sull'intero dominio convesso. In un minimo locale interno di una funzione due volte differenziabile, l'Hessiana deve essere almeno semidefinita positiva; questa condizione necessaria, però, non è sufficiente da sola. Se l'Hessiana è soltanto semidefinita, il test può essere **inconcludente** e occorrono ulteriori informazioni.

Anche in una sola dimensione, assumendo $g'(v) = 0$, valgono i criteri

$$g''(v) > 0 \quad \Longrightarrow \quad v \text{ è un minimo locale stretto,}$$
$$g''(v) < 0 \quad \Longrightarrow \quad v \text{ è un massimo locale stretto.}$$

Se $g''(v) = 0$ non è possibile classificare il punto con questo solo test: $g(w) = w^4$ ha un minimo in 0, mentre $g(w) = w^3$ ha in 0 un flesso stazionario. In entrambi i casi la derivata prima e quella seconda si annullano nel punto.

Un esempio in due dimensioni chiarisce il collegamento con gli autovalori. Per

$$g(w_1, w_2) = w_1^2 + 2w_2^2 \quad \Longrightarrow \quad \nabla^2 g = \begin{bmatrix} 2 & 0 \\ 0 & 4 \end{bmatrix},$$

l'Hessiana ha due autovalori positivi: la funzione è convessa e $(0, 0)$ è il minimo globale. Per $g(w_1, w_2) = w_1^2 - w_2^2$, invece,

$$\nabla^2 g = \begin{bmatrix} 2 & 0 \\ 0 & -2 \end{bmatrix},$$

la curvatura ha segni opposti nelle due direzioni: l'origine è un punto di sella, non un minimo né un massimo.

10.4 Funzioni quadratiche e approssimazione di Taylor

Una forma generale utile per descrivere una funzione quadratica a più variabili è

$$\boxed{g(\mathbf{w}) = a + \mathbf{b}^T\mathbf{w} + \mathbf{w}^T C\mathbf{w}}$$

dove:

- a è un termine scalare costante;
- $\mathbf{b} \in \mathbb{R}^N$ contiene i coefficienti dei termini di primo grado;
- $C \in \mathbb{R}^{N\times N}$ è una matrice, che può essere considerata simmetrica nella rappresentazione della forma quadratica;
- $\mathbf{w} \in \mathbb{R}^N$ è il vettore delle variabili o dei parametri.

In una sola dimensione la stessa struttura si riduce alla forma

$$g(w) = a + bw + cw^2,$$

cioè alla forma di un polinomio quadratico, il cui grafico è una parabola quando $c \neq 0$. In più dimensioni il termine $\mathbf{w}^T C\mathbf{w}$ rappresenta la componente quadratica della funzione e permette di descrivere il comportamento di funzioni quadratiche con più parametri, come quelle utilizzate negli esempi di ottimizzazione.

Se C è simmetrica, il gradiente e l'Hessiana della funzione quadratica sono rispettivamente

$$\nabla g(\mathbf{w}) = \mathbf{b} + 2C\mathbf{w}, \qquad \nabla^2 g(\mathbf{w}) = 2C.$$

Il termine quadratico determina quindi direttamente la curvatura: se gli autovalori di C sono tutti non negativi la funzione è convessa, mentre se sono tutti non positivi è concava. Il fattore 2 non cambia il segno degli autovalori.

La forma quadratica permette di comprendere geometricamente l'approssimazione di Taylor del **secondo ordine**: una funzione anche non quadratica viene descritta, in un intorno di un punto $\mathbf{v}$, tramite il valore della funzione, la pendenza e la curvatura in quel punto.

In una sola variabile, assumendo g due volte differenziabile nell'intorno considerato, l'approssimazione è

$$\boxed{h(w) = g(v) + g'(v)(w - v) + \frac{1}{2}g''(v)(w - v)^2.}$$

Per una funzione a N variabili, la corrispondente espressione è

$$\boxed{\begin{aligned} h(\mathbf{w}) = g(\mathbf{v}) &+ \nabla g(\mathbf{v})^T(\mathbf{w} - \mathbf{v}) \\ &+ \frac{1}{2}(\mathbf{w} - \mathbf{v})^T \nabla^2 g(\mathbf{v})(\mathbf{w} - \mathbf{v}). \end{aligned}}$$

Il primo termine fissa il valore nel punto $\mathbf{v}$, il secondo rappresenta l'approssimazione **lineare** già incontrata con Taylor di primo ordine e il terzo introduce la **curvatura** attraverso l'Hessiana. La forma del prodotto matriciale è coerente con le dimensioni:

$$\underbrace{(\mathbf{w} - \mathbf{v})^T}_{1\times N}\underbrace{\nabla^2 g(\mathbf{v})}_{N\times N}\underbrace{(\mathbf{w} - \mathbf{v})}_{N\times 1} \in \mathbb{R}.$$

L'approssimazione quadratica h coincide con g nel valore, nel gradiente e nell'Hessiana al punto di riferimento, ma *non è in generale uguale alla funzione originale lontano da quel punto*. Quando l'Hessiana nel punto di riferimento è definita positiva, l'approssimazione quadratica ha forma convessa e possiede un minimo unico; se è definita negativa, ha invece un massimo unico.

10.5 Il metodo di Newton: dall'approssimazione all'algoritmo

Il metodo di Newton sfrutta la curvatura per aggiornare i parametri in modo diverso rispetto alla discesa del gradiente. In ciascuna iterazione considera il punto corrente e costruisce un'approssimazione **quadratica di Taylor**; individua quindi il punto stazionario di tale approssimazione e lo utilizza per proporre un nuovo punto. L'idea è particolarmente utile quando la funzione presenta curvature molto diverse lungo le differenti direzioni: la sola direzione del gradiente può infatti produrre i percorsi a zig-zag già discussi.

10.5.1 Aggiornamento in una variabile

Partendo dal punto $w^{(k-1)}$, indichiamo con h_k l'approssimazione di Taylor del secondo ordine:

$$\begin{aligned} h_k(w) = g\big(w^{(k-1)}\big) &+ g'\big(w^{(k-1)}\big)(w - w^{(k-1)}) \\ &+ \frac{1}{2}g''\big(w^{(k-1)}\big)(w - w^{(k-1)})^2. \end{aligned}$$

Per individuare il punto stazionario dell'approssimazione poniamo a zero la derivata rispetto alla variabile w:

$$h_k'(w) = g'\big(w^{(k-1)}\big) + g''\big(w^{(k-1)}\big)(w - w^{(k-1)}) = 0.$$

Se $g''(w^{(k-1)}) \neq 0$, ricaviamo il successivo punto proposto:

$$\boxed{w^{(k)} = w^{(k-1)} - \frac{g'\big(w^{(k-1)}\big)}{g''\big(w^{(k-1)}\big)}.}$$

Questa espressione identifica il punto stazionario della parabola di Taylor. Se la derivata seconda nel punto corrente è positiva, la parabola è convessa e quel punto ne rappresenta il **minimo**; se è negativa, il punto è invece un massimo dell'approssimazione. Non bisogna confondere l'ottimo dell'approssimazione con quello della funzione originaria.

10.5.2 Aggiornamento in più variabili

Per una funzione $g : \mathbb{R}^N \to \mathbb{R}$ e un punto corrente $\mathbf{w}^{(k-1)}$, l'approssimazione quadratica diventa

$$h_k(\mathbf{w}) = g(\mathbf{w}^{(k-1)}) + \nabla g(\mathbf{w}^{(k-1)})^T(\mathbf{w} - \mathbf{w}^{(k-1)}) + \frac{1}{2}(\mathbf{w} - \mathbf{w}^{(k-1)})^T \nabla^2 g(\mathbf{w}^{(k-1)})(\mathbf{w} - \mathbf{w}^{(k-1)}).$$

Per trovare il punto stazionario imponiamo che il gradiente di h_k rispetto a $\mathbf{w}$ si annulli:

$$\nabla h_k(\mathbf{w}) = \nabla g(\mathbf{w}^{(k-1)}) + \nabla^2 g(\mathbf{w}^{(k-1)})(\mathbf{w} - \mathbf{w}^{(k-1)}) = \mathbf{0}.$$

Se l'Hessiana è invertibile, da questa equazione si ottiene il **passo di Newton**:

$$\boxed{\mathbf{w}^{(k)} = \mathbf{w}^{(k-1)} - \left[\nabla^2 g(\mathbf{w}^{(k-1)})\right]^{-1} \nabla g(\mathbf{w}^{(k-1)}).}$$

La presenza dell'**inversa dell'Hessiana** deriva quindi dalla risoluzione della condizione di stazionarietà dell'approssimazione quadratica, non dalla formula di Taylor in sé: *nella formula di Taylor compare l'Hessiana, senza inversione.*

Il metodo rientra nello schema generale di ottimizzazione locale incontrato in precedenza:

$$\mathbf{w}^{(k)} = \mathbf{w}^{(k-1)} + \alpha_k \mathbf{d}^{(k-1)},$$

con direzione

$$\boxed{\mathbf{d}^{(k-1)} = -\left[\nabla^2 g(\mathbf{w}^{(k-1)})\right]^{-1} \nabla g(\mathbf{w}^{(k-1)}).}$$

Nella versione classica di Newton si prende $\alpha_k = 1$; introdurre $\alpha_k > 0$ permette invece di regolare la lunghezza dello spostamento, come per gli altri metodi di ottimizzazione locale. Se l'Hessiana è **definita positiva** e il gradiente è diverso da zero, questa direzione è una direzione di discesa, poiché

$$\nabla g(\mathbf{w})^T \mathbf{d} = -\nabla g(\mathbf{w})^T [\nabla^2 g(\mathbf{w})]^{-1} \nabla g(\mathbf{w}) < 0.$$

La proprietà assicura una diminuzione del costo per passi *sufficientemente piccoli*, ma non necessariamente per un passo completo con $\alpha_k = 1$: il modello di Taylor descrive soprattutto il comportamento **locale** della funzione. Se l'Hessiana non è definita positiva, la direzione proposta potrebbe persino essere di salita o non essere definita quando l'Hessiana è singolare.

10.5.3 Perché può convergere più rapidamente

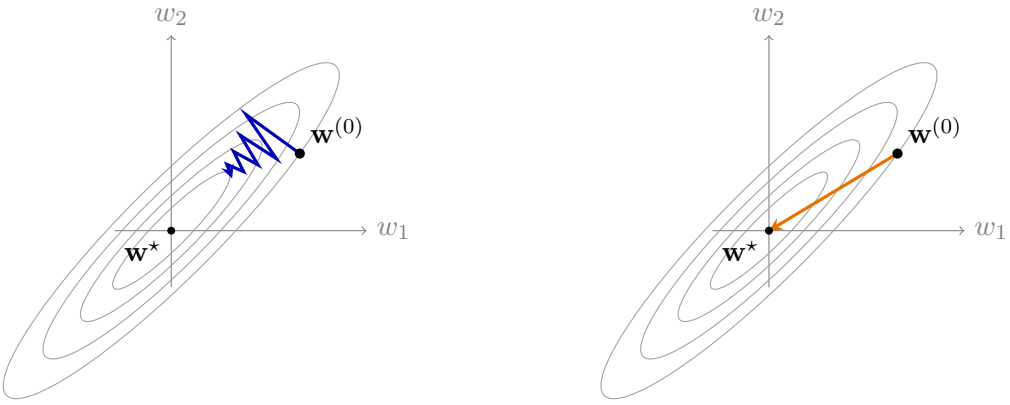
Quando la funzione è una **quadratica con Hessiana definita positiva**, l'approssimazione di Taylor del secondo ordine coincide con la funzione originale: Newton con passo completo raggiunge il minimo globale in **una sola iterazione**, indipendentemente dal punto iniziale.

Per una funzione non quadratica, invece, il metodo ricostruisce a ogni passo una diversa parabola o superficie quadratica locale. In condizioni favorevoli, soprattutto vicino a un minimo con Hessiana definita positiva e regolare, questa strategia può richiedere molte meno iterazioni della gradient descent; non garantisce tuttavia convergenza rapida o monotona da qualunque punto di partenza.

La differenza rispetto alla discesa del gradiente si può osservare sulla funzione quadratica utilizzata anche negli esempi del docente:

$$g(w_1, w_2) = 0.26(w_1^2 + w_2^2) - 0.48 w_1 w_2.$$

Le sue curve di livello sono ellissi molto allungate: la gradient descent con un passo opportuno può oscillare fra i due lati della valle e avanzare lentamente lungo di essa; Newton usa anche la curvatura e individua direttamente il minimo $\mathbf{0}$.



Gradient descent ($\alpha = 1.8$) Newton ($\alpha = 1$)

Figura 10: Confronto sulla medesima funzione quadratica, partendo da $\mathbf{w}^{(0)} = (2.5, 1.5)^T$. Le traiettorie sono ottenute dalle rispettive formule iterative: con questa scelta del learning rate, la gradient descent oscilla nella valle, mentre Newton raggiunge il minimo in un passo.

10.5.4 Costo computazionale e soluzione tramite sistemi lineari

L'espressione con l'Hessiana inversa è utile per comprendere il metodo, ma **non è necessario calcolare esplicitamente tale inversa**. A ogni passo è sufficiente risolvere un sistema lineare. Ponendo

$$H_k = \nabla^2 g(\mathbf{w}^{(k-1)}), \qquad \mathbf{q}_k = \nabla g(\mathbf{w}^{(k-1)}),$$

si può calcolare la direzione $\mathbf{d}^{(k-1)}$ risolvendo

$$\boxed{H_k \mathbf{d}^{(k-1)} = -\mathbf{q}_k}, \qquad \mathbf{w}^{(k)} = \mathbf{w}^{(k-1)} + \alpha_k \mathbf{d}^{(k-1)}.$$

Per $\alpha_k = 1$ la formulazione equivalente in termini del nuovo punto è

$$\boxed{H_k \mathbf{w}^{(k)} = H_k \mathbf{w}^{(k-1)} - \mathbf{q}_k.}$$

L'Hessiana moltiplicata per un vettore *non* produce in generale la matrice identità: l'identità si ottiene, quando l'inversa esiste, dal prodotto $H_k^{-1} H_k = I$.

Per una matrice densa di dimensione $N \times N$, un metodo diretto standard per risolvere il sistema richiede tipicamente un numero di operazioni dell'ordine di $O(N^3)$ e la memorizzazione della matrice richiede $O(N^2)$ spazio. La struttura della matrice e gli algoritmi numerici

utilizzati possono modificare questi costi; occorre inoltre considerare il lavoro necessario a calcolare l'Hessiana. Risolvere il sistema lineare è comunque preferibile, dal punto di vista numerico, al formare esplicitamente l'inversa.

10.5.5 Instabilità e regolarizzazione

Se la funzione presenta curvature molto piccole, l'Hessiana può essere singolare o quasi singolare. In una variabile, quando $g''(w)$ è vicino a zero, il rapporto $g'(w)/g''(w)$ può diventare molto grande o numericamente instabile; in più variabili, piccoli autovalori dell'Hessiana possono amplificare gli errori numerici.

Una possibile contromisura consiste nell'aggiungere un piccolo termine positivo di **regolarizzazione**. In una dimensione si considera, ad esempio, un aggiornamento della forma

$$w^{(k)} = w^{(k-1)} - \alpha_k \frac{g'\big(w^{(k-1)}\big)}{g''\big(w^{(k-1)}\big) + \varepsilon}, \qquad \varepsilon > 0.$$

Nel caso multidimensionale la corrispondente espressione è

$$\boxed{\begin{aligned}\mathbf{w}^{(k)} &= \mathbf{w}^{(k-1)} \\ &\quad - \alpha_k \big[\nabla^2 g(\mathbf{w}^{(k-1)}) + \varepsilon I\big]^{-1} \nabla g(\mathbf{w}^{(k-1)}),\end{aligned}}$$

con $I \in \mathbb{R}^{N\times N}$ matrice identità. Il termine εI agisce sugli elementi diagonali dell'Hessiana, spostandone gli autovalori di ε. Un valore positivo piccolo può migliorare la stabilità quando l'Hessiana è positiva ma quasi singolare; **non garantisce**, se l'Hessiana è indefinita, che la matrice risultante sia definita positiva. La scelta di ε dipende quindi dal problema, e non esiste un valore universale adatto a ogni funzione.

10.5.6 Vantaggi, limiti e origine del metodo

L'uso congiunto di gradiente e curvatura permette a Newton di correggere la direzione e la scala dello spostamento in funzione della geometria locale. In situazioni favorevoli questo riduce il comportamento a zig-zag tipico di alcuni metodi del primo ordine e porta a una convergenza molto rapida vicino al minimo. Per contro, un singolo passo richiede più lavoro e più memoria rispetto alla semplice valutazione del gradiente. Su funzioni non convesse il passo può puntare verso un massimo o una direzione sfavorevole, mentre un'Hessiana quasi singolare può rendere necessario adottare strategie di stabilizzazione.

Storicamente il **metodo di Newton** nasce come procedura per cercare gli zeri di una funzione; nell'ottimizzazione viene applicato alla ricerca degli zeri del gradiente, cioè delle soluzioni di $\nabla g(\mathbf{w}) = \mathbf{0}$. Questo spiega anche perché individuare un punto stazionario con Newton non significhi, automaticamente, aver trovato un minimo.

Capitolo 11

Regressione lineare e funzione di costo ai minimi quadrati

Dopo aver studiato come cercare i parametri che minimizzano una funzione, possiamo applicare questi strumenti a un problema concreto di **apprendimento supervisionato**: la regressione lineare. La regressione, introdotta fra i principali paradigmi di apprendimento, produce un valore numerico continuo; l'aggettivo *lineare* si riferisce invece alla forma scelta per il modello.

11.1 Dai dati a una previsione

Consideriamo, per esempio, un insieme di abitazioni. Per ciascuna casa conosciamo la superficie in metri quadrati e il prezzo di vendita. Possiamo rappresentare ogni osservazione come un punto nel piano: superficie sull'asse orizzontale e prezzo su quello verticale. Se i prezzi tendono ad aumentare con la superficie, possiamo cercare una retta che riassuma la tendenza dei dati e che permetta di stimare il prezzo di un'abitazione non ancora osservata.

Un problema di regressione non richiede necessariamente di prevedere *valori futuri nel tempo*: più in generale, consente di stimare un'**uscita sconosciuta** a partire dalle caratteristiche disponibili. La superficie è in questo esempio una *variabile indipendente* (feature), mentre il prezzo è la *variabile dipendente* (target) che vogliamo predire.

I dati di addestramento possono essere scritti come

$$\mathcal{D} = \{(\mathbf{x}_p, y_p)\}_{p=1}^{P}, \qquad \mathbf{x}_p \in \mathbb{R}^N, \quad y_p \in \mathbb{R}.$$

Qui P è il **numero di esempi**, mentre N è il **numero di caratteristiche** per ciascun esempio. Il pedice p identifica un particolare campione (ad esempio la casa numero 53), mentre la componente $x_{i,p}$ rappresenta la caratteristica i-esima di quel campione:

$$\mathbf{x}_p = \begin{bmatrix} x_{1,p} & x_{2,p} & \cdots & x_{N,p} \end{bmatrix}^T.$$

Se $N = 1$, i dati sono coppie di numeri (x_p, y_p) e il modello si rappresenta mediante una retta nel piano. Con più caratteristiche la stessa idea si estende a un iperpiano in uno spazio di dimensione maggiore.

11.2 Modello e ruolo dei parametri

Con una sola caratteristica la regressione lineare utilizza un modello della forma

$$\hat{y}_p = w_0 + w_1 x_p,$$

in cui w_0 è l'**intercetta** o **bias** e w_1 è il **peso** che determina la pendenza della retta. Il simbolo $\hat{y}_p$ indica la **predizione del modello**, da distinguere dal valore osservato y_p.

Con N caratteristiche la relazione diventa

$$\hat{y}_p = w_0 + \sum_{i=1}^{N} w_i x_{i,p}.$$

Per scrivere la stessa espressione in maniera compatta, aggiungiamo alla rappresentazione delle caratteristiche una prima componente costante, uguale a 1:

$$\tilde{\mathbf{x}}_p = \begin{bmatrix} 1 \\ x_{1,p} \\ \vdots \\ x_{N,p} \end{bmatrix}, \qquad \mathbf{w} = \begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_N \end{bmatrix}.$$

Il vettore $\tilde{\mathbf{x}}_p \in \mathbb{R}^{N+1}$ contiene quindi anche la componente artificiale necessaria a rappresentare il bias; non introduce una nuova caratteristica misurata sull'oggetto. Il vettore $\mathbf{w} \in \mathbb{R}^{N+1}$ contiene tutti i parametri da apprendere. Il modello può ora essere scritto come

$$\boxed{\hat{y}_p = \tilde{\mathbf{x}}_p^T \mathbf{w} \approx y_p, \qquad p = 1, \ldots, P.}$$

Il simbolo $\approx$ significa che desideriamo *avvicinare* la predizione al valore osservato: i dati reali possono contenere variabilità ed errori, quindi non è generalmente possibile ottenere un'uguaglianza esatta su tutti gli esempi.

11.3 Errore di previsione e minimi quadrati

La scelta della retta o dell'iperpiano richiede di stabilire **che cosa significhi adattarsi bene ai dati** (*fitting*). Per il singolo esempio p, definiamo il **residuo** come la differenza fra valore predetto e valore osservato:

$$r_p(\mathbf{w}) = \hat{y}_p - y_p = \tilde{\mathbf{x}}_p^T \mathbf{w} - y_p.$$

Se sommassimo semplicemente i residui, gli errori positivi e quelli negativi potrebbero annullarsi fra loro. Un criterio naturale è quindi considerare il **quadrato di ciascun residuo**, ottenendo una quantità sempre non negativa. In questo modo errori grandi pesano maggiormente rispetto a errori piccoli. Un'altra possibilità sarebbe il valore assoluto, ma il quadrato ha una forma matematica particolarmente comoda per l'ottimizzazione.

La **funzione di costo ai minimi quadrati** (*least squares*) media gli errori quadratici su tutti i P campioni:

$$\boxed{\begin{aligned} g(\mathbf{w}) &= \frac{1}{P} \sum_{p=1}^{P} g_p(\mathbf{w}) \\ &= \frac{1}{P} \sum_{p=1}^{P} \left(\tilde{\mathbf{x}}_p^T \mathbf{w} - y_p\right)^2, \end{aligned}}$$

con $g_p(\mathbf{w}) = r_p(\mathbf{w})^2$ costo del singolo esempio. Il problema di apprendimento consiste quindi nel determinare i parametri che minimizzano questa funzione:

$$\boxed{\mathbf{w}^\star \in \underset{\mathbf{w} \in \mathbb{R}^{N+1}}{\arg\min}\, g(\mathbf{w}).}$$

Si tratta di un problema di **ottimizzazione non vincolata**: la formulazione non impone restrizioni aggiuntive sui valori dei pesi. Il fattore $1/P$ permette di esprimere l'errore quadratico medio e non cambia la posizione dei minimi rispetto alla somma non divisa per P.

Per esempio, se su tre abitazioni un modello produce errori di 0, -10 e $+10$ (espressi in migliaia di euro), il costo medio quadratico è

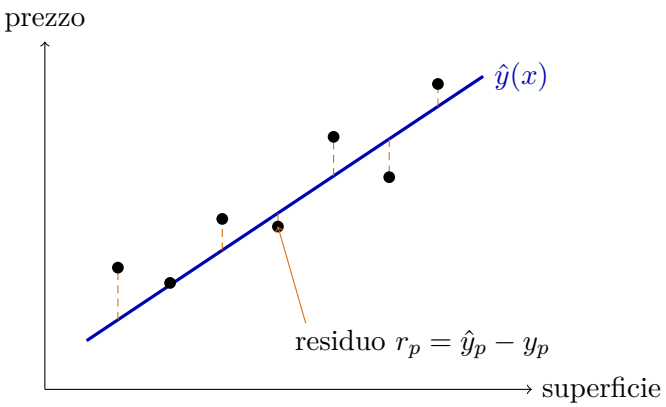


Figura 11: Regressione lineare su dati schematici: la retta rappresenta le predizioni del modello; i segmenti verticali tratteggiati indicano gli scarti rispetto ai valori osservati. La funzione ai minimi quadrati cerca i parametri che rendono piccola la somma dei quadrati di questi residui.

$$g(\mathbf{w}) = \frac{0^2 + (-10)^2 + 10^2}{3} = \frac{200}{3}.$$

Questo esempio mette in evidenza che errori di segno opposto non si compensano: conta la loro **ampiezza quadratica**.

11.4 Convessità e collegamento con le equazioni normali

La funzione di costo ai minimi quadrati è una **funzione quadratica dei parametri**, indipendentemente dal numero di campioni: ogni residuo è affine in $\mathbf{w}$ e il suo quadrato è una funzione quadratica. Il costo è quindi differenziabile e **convesso**, anche se non necessariamente strettamente convesso: dataset differenti possono non determinare un'unica scelta di pesi.

Per capire il collegamento con Newton e con le **equazioni normali**, raccogliamo gli esempi nella matrice

$$X = \begin{bmatrix} \tilde{\mathbf{x}}_1^T \\ \tilde{\mathbf{x}}_2^T \\ \vdots \\ \tilde{\mathbf{x}}_P^T \end{bmatrix} \in \mathbb{R}^{P\times(N+1)}, \qquad \mathbf{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_P \end{bmatrix}.$$

La stessa funzione obiettivo si riscrive come

$$g(\mathbf{w}) = \frac{1}{P}\|X\mathbf{w} - \mathbf{y}\|_2^2,$$

con gradiente e Hessiana

$$\nabla g(\mathbf{w}) = \frac{2}{P}X^T(X\mathbf{w} - \mathbf{y}), \qquad \nabla^2 g(\mathbf{w}) = \frac{2}{P}X^T X.$$

La formula del gradiente si ottiene derivando rispetto ai singoli parametri, oppure raccogliendo direttamente i residui nella forma matriciale. Scritta come somma sui campioni, essa è

$$\nabla g(\mathbf{w}) = \frac{2}{P}\sum_{p=1}^{P} \tilde{\mathbf{x}}_p(\tilde{\mathbf{x}}_p^T\mathbf{w} - y_p).$$

Qui $\tilde{\mathbf{x}}_p$ è un vettore colonna: moltiplicando il residuo scalare per tale vettore si ottiene un gradiente della stessa dimensione di $\mathbf{w}$. Poiché X contiene nelle righe le $\tilde{\mathbf{x}}_p^T$, la sommatoria coincide esattamente con $X^T(X\mathbf{w} - \mathbf{y})$. Questo passaggio permette di ricondurre la ricerca dei parametri a un problema di algebra lineare.

L'Hessiana è **semidefinita positiva**, perché per ogni vettore $\mathbf{z}$ si ha

$$\mathbf{z}^T(X^TX)\mathbf{z} = \|X\mathbf{z}\|_2^2 \geq 0.$$

Imponendo $\nabla g(\mathbf{w}) = \mathbf{0}$ e semplificando il fattore $2/P$ si ottiene

$$X^T(X\mathbf{w} - \mathbf{y}) = \mathbf{0} \quad \Longleftrightarrow \quad X^TX\mathbf{w} = X^T\mathbf{y}.$$

Quest'ultima forma costituisce il sistema

$$\boxed{A\mathbf{w} = \mathbf{b}, \qquad A = X^TX, \quad \mathbf{b} = X^T\mathbf{y}.}$$

Le **equazioni normali** sono quindi il sistema lineare ottenuto ponendo a zero il gradiente del costo quadratico. Si chiamano «normali» perché alla soluzione il vettore dei **residui** $\mathbf{r} = X\mathbf{w} - \mathbf{y}$ è ortogonale a ogni colonna di X, ossia $X^T\mathbf{r} = \mathbf{0}$: geometricamente, non resta alcuna componente dell'errore che possa essere eliminata modificando i pesi lungo le direzioni rappresentate dalle caratteristiche. Questa interpretazione non richiede di conoscere in dettaglio l'algebra delle proiezioni per utilizzare il risultato.

Si tratta delle condizioni di stazionarietà del costo ai minimi quadrati: poiché il costo è convesso, ogni loro soluzione è un **minimo globale**. Se le colonne di X sono linearmente indipendenti, la matrice X^TX è invertibile e la soluzione è unica:

$$\mathbf{w}^\star = (X^TX)^{-1}X^T\mathbf{y}.$$

Se invece X^TX è singolare, non si può utilizzare questa formula con l'inversa e possono esistere più soluzioni ottimali; rimane comunque valido il problema di minimi quadrati.

Il collegamento con Newton è immediato: essendo il costo esattamente quadratico e avendo Hessiana costante, **un solo passo completo di Newton raggiunge la soluzione ottimale** quando l'Hessiana è invertibile. Le equazioni normali esprimono analiticamente quello stesso punto senza dover effettuare più aggiornamenti iterativi. Questa proprietà riguarda specificamente il modello lineare con costo quadratico: non vale automaticamente per ogni problema di regressione o per ogni funzione di costo.

11.5 Outlier e funzione di costo alle deviazioni assolute

La funzione ai minimi quadrati attribuisce un peso particolarmente alto ai **residui di grande ampiezza**. Questo è utile quando vogliamo penalizzare molto le previsioni fortemente errate, ma può rendere il modello sensibile agli **outlier**, cioè osservazioni che si discostano marcatamente dal comportamento prevalente dei dati. Un prezzo di vendita inserito in modo errato, ad esempio, può influenzare sensibilmente la retta ottenuta: minimizzare i quadrati dei residui potrebbe infatti richiedere di peggiorare l'adattamento a molti campioni ordinari per ridurre l'errore su quel punto anomalo.

Questa sensibilità non va confusa con l'*overfitting*. Gli outlier riguardano le caratteristiche dei dati e la risposta della funzione di costo; l'overfitting è invece il fenomeno per cui un

modello si adatta molto ai dati di training ma generalizza male su esempi nuovi. I due fenomeni possono interagire, ma non sono sinonimi. Anche eliminare automaticamente i punti anomali è rischioso: un'osservazione insolita può essere un dato valido e significativo, non un errore da correggere.

Una possibile alternativa alla perdita quadratica è la **perdita assoluta** (*least absolute deviations*, LAD). Per il singolo esempio e per l'intero dataset si definiscono rispettivamente

$$\ell_p^{\mathrm{L1}}(\mathbf{w}) = \left|\tilde{\mathbf{x}}_p^T\mathbf{w} - y_p\right|, \qquad g_{\mathrm{L1}}(\mathbf{w}) = \frac{1}{P}\sum_{p=1}^{P}\ell_p^{\mathrm{L1}}(\mathbf{w}).$$

Il nome L1 richiama la norma $\|\mathbf{r}\|_1$ del vettore dei residui. Un errore di ampiezza 5 contribuisce con 25 al costo quadratico e con 5 a quello assoluto: rispetto alla perdita L2, quella L1 attenua l'influenza *relativa* dei residui molto grandi.

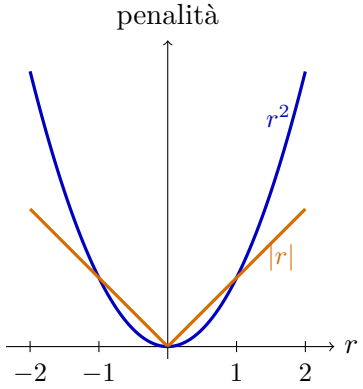


Figura 12: Confronto schematico tra perdita quadratica e assoluta in funzione del residuo r. Il quadrato cresce più rapidamente al crescere dell'ampiezza dell'errore; per piccoli residui le due curve hanno invece un comportamento diverso anche vicino a zero.

Il valore assoluto è **continuo e convesso**, ma non è **derivabile nei punti in cui il residuo è zero**. Di conseguenza, non si può applicare ovunque la normale formula della gradient descent basata su un gradiente classico, né la formula di Newton che richiede un'Hessiana. Il problema può comunque essere affrontato, per esempio, con metodi basati su *subgradienti*, con metodi senza derivate o con formulazioni specifiche di ottimizzazione convessa. La presenza del valore assoluto non rende quindi il problema irrisolvibile e non lo limita necessariamente ai soli metodi del primo ordine.

11.6 Regressione pesata

In alcuni casi non vogliamo attribuire la stessa importanza a tutte le osservazioni. Può accadere, ad esempio, che certi dati abbiano misure più affidabili di altri, oppure che abbiano rilevanza differente per l'applicazione. Se questa informazione è disponibile, introduciamo un **peso per campione** $\beta_p \geq 0$, ottenendo

$$\boxed{g_{\mathrm{pes}}(\mathbf{w}) = \frac{1}{P}\sum_{p=1}^{P}\beta_p\left(\tilde{\mathbf{x}}_p^T\mathbf{w} - y_p\right)^2.}$$

Se $\beta_p = 1$ per tutti i campioni, ritroviamo i normali minimi quadrati; un peso maggiore enfatizza il corrispondente errore, mentre un peso minore ne riduce l'influenza. Un peso nullo rende quell'esempio irrilevante per il costo. I β_p sono **pesi dei campioni**, distinti dai

parametri w_i che l'algoritmo apprende. La loro scelta deve avere una giustificazione: non possiamo sapere quali osservazioni sono poco affidabili semplicemente perché il modello vi commette un errore grande.

La regressione pesata mantiene la struttura quadratica rispetto ai parametri e, con pesi non negativi, rimane convessa. Nel caso dei minimi quadrati pesati può essere utile definire $B = \mathrm{diag}(\beta_1, \ldots, \beta_P)$; le equazioni normali assumono allora la forma

$$X^T B X \mathbf{w} = X^T B \mathbf{y}.$$

Anche questa è un'applicazione dello stesso principio: si sceglie una funzione obiettivo coerente con il problema e si cercano i pesi del modello che la minimizzano. Un'eventuale normalizzazione per $\sum_p \beta_p$, invece di P, ne cambia la scala ma non i minimizzatori se i pesi sono fissati e la somma è positiva.

Capitolo 12

Classificazione lineare binaria e regressione logistica

La regressione lineare permette di stimare una grandezza continua, mentre nella **classificazione binaria** l'obiettivo è assegnare ogni dato a una fra due classi, per esempio 0 e 1. Un modello lineare può essere utilizzato anche in questo secondo contesto: invece di interpretare direttamente il suo risultato come una previsione numerica, lo trasformiamo in una decisione di classe. Questa distinzione riprende il concetto di confine decisionale introdotto nella Sezione 4.1; nel seguito ne studiamo soprattutto il ruolo nell'**addestramento**.

12.1 Dal punteggio lineare alla funzione gradino

Per il campione p, consideriamo il vettore delle caratteristiche comprensivo della componente costante necessaria al bias, già definito nella regressione lineare:

$$\tilde{\mathbf{x}}_p = \begin{bmatrix} 1 & x_{1,p} & \cdots & x_{N,p} \end{bmatrix}^T, \qquad \mathbf{w} = \begin{bmatrix} w_0 & w_1 & \cdots & w_N \end{bmatrix}^T.$$

Il modello calcola anzitutto un **punteggio lineare**:

$$z_p = \tilde{\mathbf{x}}_p^T \mathbf{w} = w_0 + \sum_{i=1}^{N} w_i x_{i,p}.$$

Per ottenere una delle due etichette possibili, possiamo applicare una **funzione gradino** (*step function*):

$$\text{step}(z) = \begin{cases} 1 & \text{se } z \geq 0, \\ 0 & \text{se } z < 0. \end{cases} \qquad \hat{y}_p = \text{step}(z_p).$$

La scelta del valore assegnato esattamente in $z = 0$ è una convenzione: non modifica la geometria del confine decisionale. L'insieme dei punti che soddisfano

$$\tilde{\mathbf{x}}^T \mathbf{w} = 0$$

è infatti il **confine decisionale**, che separa le due regioni dello spazio delle caratteristiche. In due dimensioni il confine è una retta, mentre in generale è un iperpiano.

La soglia zero non è una limitazione: per una decisione della forma $\text{step}(\mathbf{v}^T\mathbf{x} - c)$, il valore c può essere assorbito nel bias w_0. Per esempio, scegliere una soglia di 0,5 sul punteggio lineare equivale a modificarne l'intercetta. È diverso, come vedremo, applicare una soglia di 0,5 alla *probabilità* restituita dalla sigmoide.

Un classificatore ideale assegna a ciascun esempio la classe corretta: $\hat{y}_p = y_p$ con $y_p \in \{0, 1\}$. Potremmo allora riutilizzare il costo ai minimi quadrati visto nella regressione, applicandolo alle predizioni della funzione gradino:

$$g_{\text{step}}(\mathbf{w}) = \frac{1}{P} \sum_{p=1}^{P} \big(\text{step}(\tilde{\mathbf{x}}_p^T \mathbf{w}) - y_p\big)^2.$$

Il problema è che il gradino è **discontinuo in zero** e **costante altrove**. Anche il costo risultante rimane costante in ampie regioni dello spazio dei parametri, con gradiente nullo

ovunque sia differenziabile. Di conseguenza, una tecnica basata sul gradiente non riceve informazioni utili su come modificare i pesi, anche quando alcune classificazioni sono errate. Non è la sola ambiguità sul valore della soglia a rendere difficile l'ottimizzazione: è soprattutto l'assenza di una variazione graduale del costo.

12.2 La sigmoide e il modello di regressione logistica

Per superare questo problema possiamo sostituire il gradino, durante l'addestramento, con una funzione continua e derivabile che ne approssimi la transizione. La scelta considerata a lezione è la **funzione sigmoide logistica**:

$$\boxed{\sigma(z) = \frac{1}{1 + e^{-z}}.}$$

La funzione è crescente, ha valori strettamente compresi tra 0 e 1, tende a 0 per $z \to -\infty$ e a 1 per $z \to +\infty$; inoltre $\sigma(0) = 1/2$. A differenza del gradino, il suo andamento è regolare:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)).$$

Applicata al punteggio lineare, definisce il modello di **regressione logistica**:

$$\boxed{\hat{p}_p = \sigma(\tilde{\mathbf{x}}_p^T \mathbf{w}).}$$

Qui $\hat{p}_p$ è il valore con cui il modello stima la **probabilità di appartenenza alla classe 1**; non è ancora un'etichetta discreta. La classe può essere assegnata, ad esempio, con la soglia convenzionale $\hat{p}_p \geq 0{,}5$, equivalente a $\tilde{\mathbf{x}}_p^T \mathbf{w} \geq 0$. Il **confine decisionale rimane lineare**: ciò che cambia è il modo in cui il punteggio viene interpretato e con cui viene calcolata la perdita per l'addestramento.

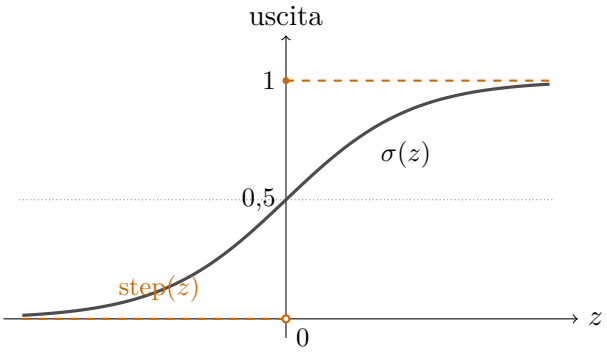


Figura 13: La sigmoide (curva continua) approssima la transizione della funzione gradino (tratteggiata) senza introdurne la discontinuità. Il valore 0,5 corrisponde a un punteggio lineare nullo.

Il termine *regressione* nel nome è storico e si riferisce alla forma parametrica del modello: **il compito affrontato è una classificazione**, non la previsione di un valore continuo qualunque. Come nel caso della regressione lineare, desideriamo determinare $\mathbf{w}$ a partire da coppie di addestramento $(\mathbf{x}_p, y_p)$. La differenza cruciale è la forma non lineare della previsione $\hat{p}_p$ rispetto ai parametri.

12.3 Perché il costo ai minimi quadrati non è sempre la scelta migliore

Possiamo inizialmente mantenere la perdita quadratica e sostituire semplicemente la funzione gradino con la sigmoide:

$$g_{\mathrm{LS}}(\mathbf{w}) = \frac{1}{P}\sum_{p=1}^{P}\Big[\sigma\big(\tilde{\mathbf{x}}_p^T\mathbf{w}\big) - y_p\Big]^2.$$

A differenza del costo costruito direttamente sul gradino, questa funzione fornisce in generale variazioni del costo utilizzabili da metodi di ottimizzazione differenziabile. **Tuttavia non è, in generale, convessa rispetto a $\mathbf{w}$**: la composizione della perdita quadratica con la sigmoide può produrre più regioni con andamento differente. Non ritroviamo, quindi, le garanzie di convessità dei minimi quadrati applicati al *modello lineare* della sezione precedente. Inoltre la sigmoide può saturare verso 0 o 1, rendendo alcuni gradienti molto piccoli.

I grafici osservati a lezione confrontano un adattamento sigmoide ottenuto tramite ottimizzazione locale e il relativo percorso sulle curve di livello della funzione di costo: illustrano che l'esito dipende dalla funzione scelta, dall'inizializzazione e dai passi dell'algoritmo. Non rappresentano una garanzia di convergenza per qualunque dataset.

12.4 Cross-entropy: una funzione di costo adatta alla classificazione binaria

In alternativa, consideriamo la **cross-entropy binaria** (*binary cross-entropy*, o *log-loss*). La perdita del singolo campione viene definita in modo da penalizzare fortemente una probabilità assegnata alla classe sbagliata:

$$\ell_p(\mathbf{w}) = \begin{cases} -\log \hat{p}_p, & y_p = 1, \\ -\log(1-\hat{p}_p), & y_p = 0, \end{cases} \qquad \hat{p}_p = \sigma(\tilde{\mathbf{x}}_p^T\mathbf{w}).$$

Le due espressioni possono essere riunite nella forma

$$\boxed{\ell_p(\mathbf{w}) = -\Big[y_p \log \hat{p}_p + (1-y_p)\log(1-\hat{p}_p)\Big].}$$

La **funzione di costo sull'intero dataset** è la media

$$\boxed{g_{\mathrm{CE}}(\mathbf{w}) = -\frac{1}{P}\sum_{p=1}^{P}\Big[y_p \log \sigma(z_p) + (1-y_p)\log\big(1-\sigma(z_p)\big)\Big], \quad z_p = \tilde{\mathbf{x}}_p^T\mathbf{w}.}$$

Se l'etichetta corretta è 1, il costo tende a zero quando $\hat{p}_p \to 1$, mentre cresce molto quando $\hat{p}_p \to 0$: si penalizza una previsione sbagliata espressa con grande sicurezza. Per $y_p = 0$ vale il comportamento simmetrico. La cross-entropy non modifica la regola geometrica di classificazione: modifica il **criterio con cui scegliamo i pesi**.

Nel caso di un **modello logistico lineare**, la cross-entropy è convessa rispetto ai parametri $\mathbf{w}$. Si può comprendere il motivo anche osservando la dipendenza dal punteggio z:

$$\frac{\mathrm{d}^2\ell}{\mathrm{d}z^2} = \sigma(z)\big(1-\sigma(z)\big) \geq 0.$$

Poiché $z = \tilde{\mathbf{x}}^T\mathbf{w}$ è affine in $\mathbf{w}$, la convessità si conserva nella composizione e nella somma dei costi. Ciò rende possibili i metodi di ottimizzazione del primo e secondo ordine già studiati; **non significa però che un algoritmo con un learning rate arbitrario converga sempre**, né che il minimizzatore debba essere necessariamente unico o finito.

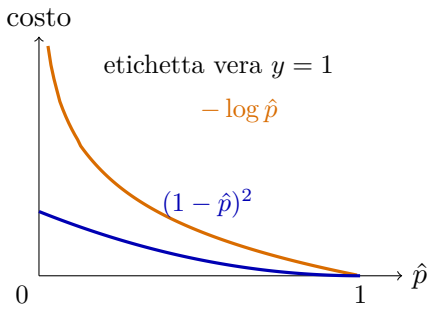


Figura 14: Per un esempio della classe 1, confronto fra perdita quadratica (blu) e logaritmica (arancione) in funzione della probabilità predetta $\hat{p}$. La log-loss cresce senza limite quando il modello assegna probabilità prossima a zero alla classe corretta; per la classe 0 il comportamento è simmetrico.

12.5 Forma softplus ed etichette meno uno e più uno

Per scrivere lo stesso problema in un'altra forma algebrica possiamo usare etichette simmetriche:

$$t_p = 2y_p - 1 \in \{-1, +1\}.$$

Con $z_p = \tilde{\mathbf{x}}_p^T \mathbf{w}$, la perdita logistica può essere riscritta, **senza modificarne il valore**, come

$$\ell_p(\mathbf{w}) = \log\big(1 + e^{-t_p z_p}\big).$$

Infatti, quando $t_p = +1$, questa espressione è $-\log \sigma(z_p)$; quando $t_p = -1$, è $-\log[1 - \sigma(z_p)]$. La funzione

$$\text{softplus}(u) = \log(1 + e^u)$$

permette quindi di scrivere $\ell_p = \text{softplus}(-t_p z_p)$. È questo il senso dell'**equivalenza algebrica** tra costo softplus e cross-entropy binaria richiamata a lezione: **non sono due criteri differenti**, ma due notazioni della stessa perdita logistica quando le etichette vengono convertite coerentemente.

Il termine *softmax* riportato in alcune slide viene impiegato anche in contesti più generali, ma non va confuso con la *softplus*: la softplus è la funzione scalare $\log(1 + e^u)$, mentre la softmax è una trasformazione di un vettore di punteggi in probabilità su più classi. Per la classificazione binaria trattata qui basta la distinzione appena introdotta; non occorre ancora sviluppare il caso multiclasse.

12.6 Tangente iperbolica e rappresentazione binaria simmetrica

Con etichette $-1, +1$, un'alternativa alla sigmoide che restituisce valori in $(0, 1)$ è la **tangente iperbolica**, che restituisce valori in $(-1, +1)$:

$$\boxed{\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} = 2\sigma(2z) - 1.}$$

La funzione è continua e derivabile, e costituisce un'approssimazione regolare della funzione segno $\text{sign}(z)$, usata per distinguere direttamente le classi -1 e $+1$. Il suo andamento può essere reso più ripido sostituendo z con az per $a > 0$; la scelta dei pesi e del bias determina la posizione e l'orientamento del confine decisionale.

È importante non confondere i ruoli: **sigmoide e tangente iperbolica sono trasformazioni del punteggio del modello**, mentre **cross-entropy e softplus descrivono la**

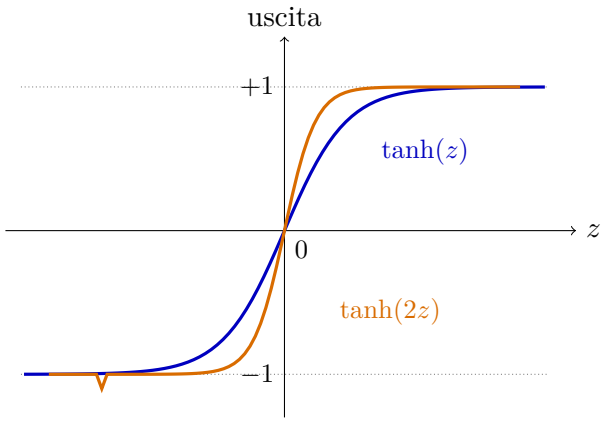


Figura 15: Tangente iperbolica e versione più ripida: entrambe sono approssimazioni continue della funzione segno, ma con valori di uscita compresi fra -1 e $+1$.

perdita da minimizzare. La tangente iperbolica non è la definizione della softplus e la sua presenza non implica, da sola, una funzione di costo specifica.

Il quadro complessivo resta quello generale dell'apprendimento supervisionato: scegliamo una famiglia di modelli, definiamo come misurarne gli errori sui dati etichettati, ottimizziamo i parametri e infine valutiamo la capacità di classificare esempi nuovi.

Capitolo 13

Generalizzazione e validazione

13.1 Generalizzazione

Una proprietà fondamentale di un modello di Machine Learning è la capacità di **generalizzare**.

La generalizzazione consiste nella capacità del modello di utilizzare ciò che ha appreso durante l'addestramento per produrre risposte adeguate anche su esempi mai utilizzati durante il training.

L'obiettivo dell'apprendimento non è quindi semplicemente memorizzare gli esempi osservati, ma individuare regolarità utilizzabili anche su nuovi dati.

13.2 Training set, validation set e test set

I dati disponibili possono essere suddivisi in insiemi distinti, con funzioni differenti all'interno del processo di apprendimento.

In particolare:

- il **training set** viene utilizzato per addestrare il modello;
- il **validation set** contiene dati non utilizzati direttamente per apprendere i parametri e permette di valutare il comportamento del modello durante il processo di sviluppo;
- il **test set** contiene dati tenuti separati dal processo di sviluppo, da usare per una valutazione finale e quanto più possibile indipendente delle prestazioni.

Il training determina i parametri del modello; la validation può guidare le scelte durante lo sviluppo, per esempio fra diversi iperparametri; il test serve a stimare le prestazioni dopo queste decisioni. Non è quindi opportuno usare ripetutamente il test per scegliere il modello, perché si perderebbe la sua funzione di controllo indipendente.

13.3 Predizione e metriche di valutazione per la regressione

Terminato l'addestramento, fissiamo il vettore dei parametri $\mathbf{w}^\star$. Il modello può quindi produrre una stima per un nuovo input $\mathbf{x}_0$: nel caso lineare,

$$\hat{y}_0 = \text{model}(\mathbf{x}_0; \mathbf{w}^\star) = \tilde{\mathbf{x}}_0^T \mathbf{w}^\star.$$

Una buona stima non coincide necessariamente con il valore reale y_0: l'obiettivo è avere un errore contenuto anche sui dati non utilizzati per apprendere i parametri. Per verificare questa capacità, si utilizzano **metriche di valutazione** calcolate su un insieme di esempi con valori osservati disponibili. Nel seguito indichiamo con M il numero di esempi valutati, distinto dal numero P di campioni usati per addestrare il modello.

Una metrica comune è il **Mean Squared Error** (MSE), o **errore quadratico medio**:

$$\boxed{\text{MSE} = \frac{1}{M} \sum_{p=1}^{M} \big(\text{model}(\mathbf{x}_p; \mathbf{w}^\star) - y_p\big)^2.}$$

Un'altra metrica è la media degli errori assoluti, **Mean Absolute Error** (MAE), detta anche *mean absolute deviation* (MAD) in alcuni materiali:

$$\boxed{\mathrm{MAE} = \frac{1}{M}\sum_{p=1}^{M} \left|\mathrm{model}(\mathbf{x}_p; \mathbf{w}^\star) - y_p\right|.}$$

L'acronimo *MAD* può indicare anche la *median absolute deviation*, una statistica diversa: è perciò opportuno precisare sempre quale definizione si usa. Le due metriche misurano lo stesso fenomeno, ma il MSE dà maggiore peso agli errori di ampiezza elevata; la MAE li pesa linearmente. Un valore più basso indica prestazioni migliori rispetto alla metrica adottata, purché il confronto avvenga sugli stessi dati e con la stessa scala del target.

È possibile usare anche la **radice dell'errore quadratico medio**, $\mathrm{RMSE} = \sqrt{\mathrm{MSE}}$, espressa nelle stesse unità della variabile da predire. La radice quadrata *non elimina la sensibilità agli outlier*: poiché è monotona crescente, non cambia l'ordinamento dei modelli rispetto al MSE calcolato sullo stesso insieme di valutazione. Se vogliamo ridurre l'influenza dei grandi errori sulla misura, scegliamo piuttosto una metrica come la MAE, secondo l'obiettivo della valutazione.

Per esempio, se i residui su tre campioni di test sono 0, 2 e 8, si ottiene

$$\mathrm{MSE} = \frac{0^2 + 2^2 + 8^2}{3} = \frac{68}{3}, \qquad \mathrm{MAE} = \frac{|0| + |2| + |8|}{3} = \frac{10}{3}.$$

La stessa formula dei minimi quadrati può dunque avere due ruoli: **funzione di costo**, quando è calcolata sul training set per scegliere i parametri, e **metrica di valutazione**, quando è calcolata su dati separati per verificare come generalizza il modello. Questa distinzione è fondamentale: un costo di training basso, da solo, non dimostra che le predizioni su nuovi esempi siano affidabili.

Capitolo 14

Pipeline di base di un sistema di Machine Learning

I concetti introdotti possono essere collegati in una pipeline generale.

1. **Raccolta e selezione dei dati**
   Vengono raccolti dati significativi per il problema da affrontare.
2. **Suddivisione dei dati**
   I dati vengono separati, ad esempio, in training set, validation set e, quando previsto, test set.
3. **Rappresentazione dei dati**
   Vengono definite le caratteristiche (*features*) attraverso cui rappresentare gli esempi.
4. **Scelta del modello**
   Viene scelta una famiglia di modelli adatta al problema.
5. **Definizione del criterio di ottimizzazione**
   Si stabilisce una funzione che permetta di valutare la qualità dei parametri del modello.
6. **Addestramento**
   Attraverso un algoritmo di ottimizzazione vengono cercati valori dei parametri $\mathbf{w}$ che riducano il costo o l'errore del modello.
7. **Validazione e valutazione**
   La qualità del modello viene analizzata su dati non usati per apprenderne i parametri. Quando previsto, un test set separato permette un controllo conclusivo dopo le scelte di sviluppo.

14.1 Miglioramento delle prestazioni

Se le prestazioni ottenute non sono soddisfacenti, è possibile intervenire su differenti elementi del sistema.

Ad esempio:

- raccogliere una maggiore quantità di dati;
- progettare caratteristiche maggiormente discriminanti;
- modificare la rappresentazione dei dati;
- utilizzare un differente tipo di modello.

Capitolo 15

Laboratorio: notebook e richiami di Python

Le esercitazioni del corso utilizzano il linguaggio Python e l'ambiente dei *notebook*, che permette di affiancare codice eseguibile, spiegazioni e risultati nello stesso documento. Questa prima parte introduce gli strumenti e richiama le caratteristiche di Python necessarie per comprendere gli esempi di laboratorio successivi. Non costituisce un corso autonomo di programmazione: l'obiettivo è poter leggere, eseguire e modificare con consapevolezza il codice utilizzato nelle esercitazioni di Machine Learning.

15.1 Notebook, Jupyter e Google Colab

Un notebook è organizzato in **celle**. Le celle di codice contengono istruzioni Python eseguite da un *kernel*; le celle Markdown contengono invece testo, formule e spiegazioni. I notebook Python hanno comunemente estensione `.ipynb` e possono essere aperti in ambienti diversi.

Jupyter Notebook permette di lavorare sul proprio computer, utilizzando un interprete Python e le librerie installate nell'ambiente locale. **Google Colab** offre un'interfaccia analoga attraverso il browser, con un ambiente di esecuzione remoto. Il formato `.ipynb` facilita il passaggio tra i due strumenti; occorre però verificare la disponibilità delle librerie, dei file e degli eventuali dati utilizzati dal notebook.

La differenza più importante rispetto a un normale file `.py` riguarda l'**ordine di esecuzione**. Il kernel conserva le variabili e le funzioni definite nelle celle già eseguite: eseguire la terza cella prima della prima, oppure rieseguire più volte una cella che modifica una lista, può produrre risultati differenti. Per controllare la riproducibilità è utile riavviare il kernel ed eseguire tutte le celle in ordine.

In una cella, il valore dell'ultima espressione viene normalmente mostrato automaticamente. Per visualizzare esplicitamente un valore è invece possibile utilizzare `print()`:

```
x = [10, 20, 30]
x[1]        # Nel notebook visualizza 20
print(x)    # Stampa [10, 20, 30]
```

L'accesso `x[1]` restituisce il secondo elemento: in Python gli indici partono da zero.

15.2 Elementi essenziali del linguaggio

Python è un linguaggio ad alto livello con tipizzazione dinamica: il tipo è associato ai valori e viene controllato durante l'esecuzione, senza richiedere una dichiarazione preventiva del tipo di ogni variabile. Tra i tipi di base figurano interi (`int`), numeri in virgola mobile (`float`), booleani (`bool`) e stringhe (`str`).

Le operazioni aritmetiche includono `+`, `-`, `*`, `/` e `**`, dove quest'ultimo operatore indica l'elevamento a potenza. Gli operatori logici si scrivono `and`, `or` e `not`. Per l'incremento di una variabile si utilizza, ad esempio, `x += 1`, non `x++`.

```
x = 3
```

```
print(x ** 2)           # 9
x += 1
print(x)                # 4
print(x > 0 and x < 10) # True

nome = "Ada"
print(f"Ciao, {nome}")   # Ciao, Ada
```

L'esempio introduce anche le *f-string*, che permettono di inserire espressioni all'interno di una stringa. Il notebook della lezione mostra inoltre alcuni metodi sulle stringhe, come `upper()`, `replace()` e `strip()`, utili rispettivamente per trasformare lettere in maiuscolo, sostituire sottostringhe e rimuovere spazi iniziali e finali.

I blocchi di codice sono delimitati dall'**indentazione**, non da parentesi graffe. Questa regola vale, ad esempio, per condizioni, cicli, funzioni e metodi delle classi.

15.3 Strutture dati e iterazione

Python fornisce diverse strutture dati integrate, ciascuna appropriata a usi differenti. Le **liste** conservano elementi ordinati e modificabili; le **tuple** sono sequenze ordinate non modificabili; i **dizionari** associano chiavi a valori; gli **insiemi** (*set*) raccolgono elementi distinti e non garantiscono un ordine di iterazione.

Una lista permette accesso per indice, inserimenti e selezione di porzioni (*slicing*):

```
valori = [3, 1, 2]
print(valori[0])       # 3
print(valori[-1])      # 2: ultimo elemento
print(valori[0:2])     # [3, 1]: estremo finale escluso
valori.append(4)
print(valori)          # [3, 1, 2, 4]
```

Le liste sono **mutabili**: una funzione può modificarne il contenuto anche senza restituire una nuova lista. Questo è il punto illustrato dal primo esempio del notebook, nel quale una funzione riceve una lista e usa `append()`:

```
def duplica_secondo_elemento(elementi):
    elementi.append(elementi[1])

numeri = [2, 3, 4]
duplica_secondo_elemento(numeri)
print(numeri)           # [2, 3, 4, 3]
```

L'argomento `elementi` fa riferimento alla stessa lista di `numeri`; `append()` modifica quindi l'oggetto originale. Non bisogna confondere questa operazione con la creazione di una nuova lista.

Per iterare si utilizza spesso il ciclo `for`. La chiamata `range(5)` produce un oggetto iterabile che rappresenta i valori da 0 a 4; se serve una lista esplicita si usa `list(range(5))`. Le **list comprehension** costituiscono una forma compatta per costruire nuove liste applicando trasformazioni e filtri:

```
numeri = [0, 1, 2, 3, 4]
quadrati_pari = [n ** 2 for n in numeri if n % 2 == 0]
print(quadrati_pari)     # [0, 4, 16]
```

La stessa operazione potrebbe essere scritta mediante un ciclo, una condizione `if` e chiamate a `append()`. Conoscere entrambe le forme è utile per leggere codice che elabora collezioni di dati.

Un dizionario associa chiavi a valori e permette di cercarli senza dover conoscere la loro posizione:

```
valutazioni = {"modello_A": 0.85, "modello_B": 0.91}
print(valutazioni["modello_A"])       # 0.85
print(valutazioni.get("modello_C", 0)) # 0
```

Il metodo `get()` permette di indicare un valore predefinito quando la chiave non è presente; l'accesso diretto con parentesi quadre genererebbe invece un `KeyError`. Per gli insiemi, un'operazione caratteristica è `add()`, che non duplica gli elementi già presenti. Una tupla come `(1, 2)` può essere utilizzata anche come chiave di un dizionario, purché tutti i suoi elementi siano a loro volta utilizzabili come chiavi.

15.4 Funzioni e classi

Le funzioni si definiscono mediante `def`; possono ricevere parametri, restituire valori con `return` e prevedere argomenti opzionali.

```
def segno(x):
    if x > 0:
        return "positivo"
    elif x < 0:
        return "negativo"
    return "zero"

print(segno(-2))       # negativo
```

Nel notebook compare anche il **QuickSort**, implementato con funzioni ricorsive e list comprehension. L'esempio è utile soprattutto perché mostra come combinare costrutti già noti: un caso base termina la ricorsione, mentre gli altri casi suddividono gli elementi rispetto a un pivot. In questa prima esercitazione non è necessario approfondirne l'analisi algoritmica.

Le classi permettono di raggruppare stato e comportamento. Il metodo speciale `__init__()` inizializza un'istanza; il parametro `self` indica l'oggetto su cui viene richiamato un metodo.

```
class Studente:
    def __init__(self, nome):
        self.nome = nome

    def presenta(self):
        return f"Sono {self.nome}"

studente = Studente("Ada")
print(studente.presenta())   # Sono Ada
```

Per il laboratorio di Machine Learning non è necessario anticipare qui un approfondimento generale sulla programmazione orientata agli oggetti: sarà sufficiente riconoscere la creazione di istanze e l'invocazione dei metodi quando compariranno nelle librerie utilizzate nel corso.

15.5 Uso del materiale nelle esercitazioni successive

Il notebook della prima esercitazione costituisce un **richiamo agli strumenti di programmazione**, non un'implementazione di algoritmi di Machine Learning. La sua utilità è preparare alla lettura del codice utilizzato nelle esercitazioni successive, nelle quali le strutture dati, le funzioni e le librerie verranno applicate a problemi concreti.

Per riprodurre gli esempi conviene conservare il notebook originale in formato `.ipynb` ed eseguirlo in ordine, controllando il comportamento delle celle che modificano variabili condivise. Le sezioni successive del laboratorio potranno approfondire gli strumenti effettivamente utilizzati, senza ripetere questa introduzione a Python.

Capitolo 16

NumPy: rappresentazione e calcolo su array

La seconda esercitazione introduce **NumPy**, una libreria per il calcolo numerico che consente di rappresentare vettori, matrici e array multidimensionali e di eseguire operazioni su di essi senza dover scrivere esplicitamente un ciclo Python per ogni elemento. Questo passaggio è rilevante per il Machine Learning: le caratteristiche dei dati, i parametri dei modelli e molte operazioni viste nella teoria vengono espressi proprio attraverso vettori e matrici.

16.1 Gli array e la loro forma

L'oggetto fondamentale della libreria è `ndarray`, un array i cui elementi hanno un tipo di dato comune (*dtype*). A differenza delle liste Python, progettate per contenere oggetti anche eterogenei, un array NumPy rappresenta in modo efficiente dati numerici omogenei. Lo si crea, per esempio, utilizzando `np.array()`:

```
import numpy as np

v = np.array([1, 2, 3])
A = np.array([[1, 2, 3], [4, 5, 6]])

print(v.shape)  # (3,)
print(A.shape)  # (2, 3)
print(A.ndim)   # 2
```

La proprietà `shape` restituisce una tupla contenente la lunghezza dell'array lungo ciascun asse; `ndim` indica il numero di assi. Il primo esempio è un array a *una* dimensione con tre elementi, mentre il secondo è un array a due dimensioni con due righe e tre colonne.

Occorre distinguere il numero di dimensioni di `ndarray` dal **rango di una matrice** in algebra lineare: nel notebook il termine inglese *rank* viene utilizzato talvolta per indicare il numero di assi, ma il rango matematico è un'altra proprietà.

Un'altra distinzione essenziale riguarda le forme dei vettori:

```
v = np.array([1, 2, 3])
riga = v.reshape(1, 3)
colonna = v.reshape(3, 1)

print(v.shape)       # (3,)
print(riga.shape)    # (1, 3)
print(colonna.shape) # (3, 1)
```

In NumPy, `(3,)` *non* è automaticamente un vettore riga di forma `(1, 3)`: è un array monodimensionale, privo di un asse esplicito che distingua riga e colonna. Questa differenza diventa importante nei prodotti matriciali. `reshape()` restituisce un array con una nuova

forma, senza modificare la proprietà `shape` della variabile originale; a seconda del caso, il risultato può condividere oppure copiare i dati sottostanti.

Per inizializzare array di uso comune, NumPy offre `np.zeros()`, `np.ones()`, `np.full()` e `np.eye()`, che produce una matrice identità. `np.random.random()` genera valori casuali uniformi nell'intervallo $[0, 1)$; `np.random.randn()` genera campioni dalla distribuzione normale standard. Per ottenere campioni normali con media μ e deviazione standard σ si può scrivere $\mu + \sigma Z$, dove Z è normale standard.

16.2 Indicizzazione, slicing e copie

Sugli array multidimensionali si accede a un elemento specificando gli indici di tutti gli assi, ad esempio `A[1, 2]` per leggere l'elemento della seconda riga e terza colonna. Come nelle liste Python, anche NumPy supporta lo *slicing*; in questo caso si può indicare una porzione per ciascuna dimensione:

```
A = np.array([[1, 2, 3, 4],
              [5, 6, 7, 8],
              [9, 10, 11, 12]])

B = A[:2, 1:3]
print(B)             # [[2 3]
                     #  [6 7]]
```

Un aspetto importante del comportamento di NumPy è la differenza fra **vista** (*view*) e **copia**. Lo slicing ordinario produce generalmente una vista sugli stessi dati dell'array originale: modificare la vista modifica quindi anche l'array di partenza.

```
B[0, 0] = 77
print(A[0, 1])  # 77

C = A[:2, 1:3].copy()
C[0, 0] = 99
print(A[0, 1])  # ancora 77
```

Non tutte le selezioni producono però viste. L'**indicizzazione avanzata** con array di interi e quella con maschere booleane restituiscono normalmente nuovi array contenenti gli elementi selezionati:

```
A = np.array([[1, 2], [3, 4], [5, 6]])
print(A[[0, 1, 2], [0, 1, 0]])  # [1 4 5]
print(A[A > 2])                 # [3 4 5 6]
```

Nel primo caso i due array di indici selezionano rispettivamente le coppie di posizioni $(0, 0)$, $(1, 1)$ e $(2, 0)$. Nel secondo, l'espressione `A > 2` genera una maschera di valori `True`/`False` usata per filtrare gli elementi. Questo meccanismo è particolarmente utile per applicare condizioni a interi insiemi di dati.

16.3 Tipi numerici e operazioni sugli elementi

Il tipo numerico di un array è osservabile con `dtype` e può essere specificato durante la creazione, ad esempio con `np.array([1, 2], dtype=np.float64)`. Il tipo incide sulla rappresentazione e sulle operazioni disponibili; è quindi utile controllarlo quando si costruiscono dati numerici a partire da sorgenti differenti.

Le operazioni aritmetiche fra array NumPy della stessa forma sono di norma applicate **elemento per elemento**. Per esempio:

```
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

print(A + B)  # [[ 6  8]
              #  [10 12]]
print(A * B)  # [[ 5 12]
              #  [21 32]]
```

L'operatore `*` non rappresenta dunque la moltiplicazione tra matrici dell'algebra lineare: calcola il prodotto degli elementi corrispondenti. Analogamente, `/` effettua una divisione elemento per elemento, mentre funzioni come `np.sqrt()` agiscono sui singoli valori. Con dimensioni compatibili, NumPy può anche applicare il *broadcasting*, estendendo logicamente un operando per eseguire l'operazione senza duplicare esplicitamente tutti i dati.

16.4 Prodotti matriciali e operazioni di algebra lineare

Il **prodotto matriciale** si calcola con l'operatore `@`, con `np.matmul()` o, nei casi usuali mostrati nel notebook, con `np.dot()`:

```
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])
print(A @ B)  # [[19 22]
              #  [43 50]]
```

Il risultato differisce da `A * B`: ciascun elemento del prodotto matriciale nasce da una somma di prodotti fra righe e colonne. Per due vettori monodimensionali, `np.dot(v, w)` restituisce il loro prodotto scalare. La trasposta di una matrice si ottiene invece mediante `A.T` oppure `A.transpose()`.

NumPy permette di aggregare valori con `np.sum()`; l'argomento `axis` individua l'asse da eliminare mediante l'aggregazione. Per una matrice, `sum(axis=0)` restituisce le somme delle colonne e `sum(axis=1)` quelle delle righe. Analogamente sono disponibili `mean()`, `std()`, `min()`, `max()`, `argmin()` e `argmax()`, utili per riassumere dati o identificare posizioni di valori estremi.

Per le operazioni più strettamente legate all'algebra lineare si utilizza il modulo `np.linalg`. Il notebook mostra, in particolare, `np.linalg.inv()` per l'inversa di una matrice quadrata invertibile, `np.linalg.eig()` per autovalori e autovettori e `np.linalg.solve()` per i sistemi lineari:

```
A = np.array([[3., 1.], [1., 2.]])
b = np.array([9., 8.])

x = np.linalg.solve(A, b)
print(x)                # [2. 3.]
print(A @ x)            # [9. 8.]

valori, vettori = np.linalg.eig(A)
# vettori[:, i] e' associato a valori[i]
```

Il comando `np.linalg.solve(A, b)` risolve direttamente $A\mathbf{x} = \mathbf{b}$, senza richiedere di calcolare esplicitamente A^{-1}. È un collegamento immediato con la teoria: il metodo di Newton e le equazioni normali della regressione possono essere ricondotti a sistemi lineari. Quando il sistema ammette una soluzione unica, risolverlo direttamente è in generale preferibile all'uso esplicito dell'inversa per ragioni di efficienza e stabilità numerica.

La funzione `np.linalg.eig(A)` restituisce un array di autovalori e una matrice le cui **colonne** sono gli autovettori corrispondenti. Questo permette di verificare operativamente la relazione $A\mathbf{v} = \lambda\mathbf{v}$ già incontrata nello studio dell'Hessiana.